

A Project Report on
PocketLancer: A MERN-Based On-Demand
Freelance and Home Services Marketplace
Submitted in partial fulfillment for the award of
Bachelor of Technology
Degree
in
Computer Science & Engineering

By

S. Dedeep Reddy (Y22ACS553) M. Siva Sai Krishna (Y22ACS497)
T. Swetha (Y22ACS576) S. Sravanthi (Y22ACS563)



Under the Guidance of
Smt. Ch. Mangamma, MTech
Assistant Professor
Department of Computer Science & Engineering
Bapatla Engineering College (Autonomous)
Affiliated to Acharya Nagarjuna University
Bapatla – 522102, Andhra Pradesh, India
2025-2026

**Department of
Computer Science and Engineering**



CERTIFICATE

This is to certify that the project report entitled **PocketLancer: A MERN-Based On-Demand Freelance and Home Services Marketplace** submitted by S. Dedee Reddy (Y22ACS553), M. Siva Sai Krishna (Y22ACS497), T. Swetha (Y22ACS576), S. Sravanthi (Y22ACS563) in partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science & Engineering is a record of bonafide work carried out under our guidance and supervision.

Date:

Guide Signature
CH. Mangamma
Assistant Professor

Head of Department
Dr. M. Rajesh Babu
Assoc. Prof. & Head

DECLARATION

We declare that this project work is composed by ourselves, that the work contained herein is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

S. Dedeep Reddy (Y22ACS553)

M. Siva Sai Krishna (Y22ACS497)

T. Swetha (Y22ACS576)

S. Sravanthi (Y22ACS563)

Acknowledgement

We sincerely thank the following distinguished personalities who have offered their valuable advice and support for the successful completion of this work.

We are deeply indebted to our most respected guide, **Smt. Ch. Mangamma, Assistant Professor, Department of CSE**, for her valuable guidance, constant encouragement, constructive suggestions, and inspiring support throughout the course of this project.

We extend our sincere thanks to **Dr. M. Rajesh Babu, Associate Professor & Head of the Department**, Department of Computer Science and Engineering, for his cooperation and for providing the necessary facilities and resources to carry out this work successfully.

We would also like to express our heartfelt gratitude to our beloved **Principal, Dr. B. Srinivasarao**, for providing the required infrastructure, online resources, and institutional support to complete this project.

We express our sincere thanks to our **Project Coordinators, Dr. C. Prakasa Rao, Professor, Department of CSE, and Dr. R. Venkata Lakshmi, Assistant Professor, Department of CSE**, for their valuable suggestions and guidance in preparing and presenting this document.

We are always thankful to our **parents and family members** for their continuous support, encouragement, and motivation. Without their support, we would not have been able to complete this project work successfully.

Finally, we extend our sincere gratitude to all the teaching faculty and non-teaching staff of the Department of Computer Science and Engineering who helped us directly or indirectly with their cooperation and encouragement.

S. Dedee Reddy (Y22ACS553)

M. Siva Sai Krishna (Y22ACS497)

T. Swetha (Y22ACS576)

S. Sravanthi (Y22ACS563)

Abstract

PocketLancer is a production-grade, full-stack web and mobile marketplace that unifies the fragmented Indian freelance and home services economy. Built on the MERN stack (MongoDB, Express.js, React/Next.js 14, Node.js), the platform serves two service categories: digital freelancing (web development, design, content writing) and on-demand field services (plumbing, electrical, AC repair, cleaning). Each category employs an appropriate payment model — milestone-based escrow via Razorpay for digital work, and an Honor Score-enforced trust framework for field services.

Key technical features: JWT authentication with OTP email verification (Resend API); MongoDB 2dsphere geospatial indexing with Leaflet.js interactive maps for nearby freelancer discovery; real-time private chat via Socket.IO with Cloudinary image sharing; Firebase Cloud Messaging (FCM) push notifications with Capacitor device token management; Razorpay milestone escrow with HMAC-SHA256 signature verification; portfolio management (images, videos, website links); review and rating engine with automatic average recalculation; FullCalendar.js availability management; structured dispute resolution with admin oversight and Cloudinary-hosted evidence; automated background jobs (node-cron) for no-show detection and dispute SLA; dark/light theme system; and Android/iOS packaging with Capacitor v6.

The system is deployed on Vercel (frontend), Render.com (backend), MongoDB Atlas (database), and Cloudinary (CDN). The platform is live at pocketlancer.org with the Android APK available for download.

Keywords: MERN Stack, Next.js 14, Socket.IO, Razorpay Escrow, Firebase FCM, MongoDB 2dsphere, Leaflet.js, Capacitor, Honor Score, Dispute Resolution

Table of Contents

List of Tables	x
List of Figures	xi
List of Symbols, Abbreviations	xii
1 Introduction.....	1
1.1 Motivation.....	1
1.2 Problem Definition.....	2
1.3 Objective of the Project	2
2 Literature Survey.....	3
2.1 Existing System	5
2.2 Disadvantages of Existing System.....	5
2.3 Proposed System.....	6
2.4 Advantages of Proposed System.....	6
3 System Analysis.....	7
3.1 Software Requirement Specification	7
3.1.1 User Requirements.....	7
3.1.2 Software Requirements.....	8
3.1.3 Hardware Requirements.....	10
3.2 System Architecture Diagram.....	10
4 Design	12
4.1 Introduction.....	12
4.2 System Design Diagrams.....	13
4.2.1 System Architecture Diagram.....	13
4.2.2 Entity Relationship Diagram.....	14

4.2.3 Booking State Diagram.....	16
4.2.4 Payment Workflow Diagram.....	17
4.2.5 Dispute Resolution Diagram.....	18
4.3 Data Dictionary.....	19
4.3.1 User Collection	19
4.3.2 Freelancer Collection	20
4.3.3 Booking Collection	21
4.3.4 Supporting Collections.....	22
4.3.5 Honor Score System	23
5 Implementation	24
5.1 Method of Implementation	24
5.1.1 Express Server & Middleware Chain	24
5.1.2 Authentication & Authorization	26
5.1.3 User & Freelancer Profile Management	28
5.1.4 Search & Geolocation Module.....	29
5.1.5 Booking & Service Management Module	31
5.1.6 Payment & Escrow Module	32
5.1.7 Real-Time Chat System	34
5.1.8 Notification & Honor Score System	35
5.1.9 Dispute Resolution System	37
5.1.10 Admin Dashboard & Analytics.....	39
5.2 System Output and Screenshots.....	40
5.2.1 Landing Page and Registration	40
5.2.2 Dashboards.....	41

5.2.3 Search and Map Discovery	42
5.2.4 Booking Creation Interface.....	42
5.2.5 Booking and Milestone Payment.....	43
5.2.6 Admin Dispute Resolution.....	43
5.2.7 Chat and Communication Interface	44
5.2.8 Notification System Interface	44
5.2.9 Freelancer Profile Editing Interface.....	45
5.2.10 Portfolio Management Interface	46
5.2.11 Admin Dashboard	46
5.2.12 Dispute Resolution Panel.....	47
5.2.13 Mobile Application Screens.....	47
5.3 Results and Analysis	48
6 Testing & Validation	50
6.1 Features to be Tested	50
6.1.1 User Authentication and Authorization	50
6.1.2 Freelancer and Profile Management	50
6.1.3 Search and Geolocation	51
6.1.4 Booking and Scheduling System	51
6.1.5 Payment Processing System	51
6.1.6 Real-Time Communication.....	51
6.1.7 Notification System	51
6.1.8 Admin Controls and Security.....	52
6.1.9 Background Jobs and Automation.....	52

6.2 Features not to be Tested	52
6.2.1 Third-Party Service Internals	52
6.2.2 Network Infrastructure	52
6.2.3 Device-specific Hardware Behavior	52
6.2.4 Large-Scale Load Testing	53
6.3 Testing Tools and Environment	53
6.4 Design of Test Cases and Scenarios	54
7 Conclusion & Future Work	56
7.1 Conclusion	56
7.2 Future Work	56
7.2.1 AI-Powered Semantic Search and Recommendation	56
7.2.2 Live Freelancer Location Tracking	57
7.2.3 React Native Rebuild	57
7.2.4 Subscription and Commission Model	57
8 References	58

List of Tables

Table 4.1	User Collection — Schema fields	19
Table 4.2	Freelancer Collection — Schema fields	20
Table 4.3	Booking Collection — Schema fields	21
Table 4.4	Supporting Collections Summary	22
Table 4.5	Honor Score Events and Penalties	23
Table 5.1	API and System Performance Benchmarks	49
Table 6.1	Test Scenarios and Results	55

List of Figures

Figure 3.1	PocketLancer Three-Tier Deployment Architecture	10
Figure 4.1	System Architecture Diagram	13
Figure 4.2	Entity Relationship Diagram	15
Figure 4.3	Booking Status State Machine Diagram	16
Figure 4.4	Payment Workflow Diagram	17
Figure 4.5	Dispute Resolution State Machine Diagram	18
Figure 5.1	Authentication Work Flow	27
Figure 5.2	PocketLancer Landing Page.....	40
Figure 5.3	Client Dashboard with Honor Score Indicator.....	41
Figure 5.4	Freelancer Dashboard — Today's Field Jobs Widget.....	41
Figure 5.5	Search Results and Map-Based Discovery.....	42
Figure 5.6	Service Booking Form.....	42
Figure 5.7	Booking Detail — Milestone Payment Flow.....	43
Figure 5.8	Admin Dispute Resolution.....	43
Figure 5.9	Chat Window.....	44
Figure 5.10	Notification Panel.....	44
Figure 5.11	Freelancer Profile Editing Screen.....	45
Figure 5.12	Digital Portfolio for Freelancers.....	46
Figure 5.13	Admin Dashboard.....	46
Figure 5.14	Admin Dispute Resolution Panel.....	47
Figure 5.15	Mobile Application Interface.....	47

List of Symbols, Abbreviations

MERN MongoDB, Express.js, React, Node.js

JWT JSON Web Token

OTP One-Time Password

FCM Firebase Cloud Messaging

SSR Server-Side Rendering

CSR Client-Side Rendering

API Application Programming Interface

REST Representational State Transfer

CDN Content Delivery Network

UPI Unified Payments Interface

INR Indian Rupee

GPS Global Positioning System

HMAC Hash-based Message Authentication Code

SLA Service Level Agreement

1 INTRODUCTION

PocketLancer is a full-stack web and mobile marketplace designed to unify India's fragmented freelance and home services ecosystem. The platform enables users to access both digital services (such as web development, design, and content writing) and on-demand field services (such as plumbing, electrical work, and appliance repair) within a single system.

Built using the MERN stack (MongoDB, Express.js, React/Next.js, and Node.js), PocketLancer provides a scalable and efficient solution for real-time service discovery, communication, and secure transactions. The platform integrates modern technologies such as geolocation-based search, real-time messaging, and milestone-based payment systems to enhance user experience and trust.

Unlike existing platforms that focus only on a single service category, PocketLancer offers a unified environment supported by INR-based payments, GPS-enabled discovery, and a structured trust mechanism.

1.1 MOTIVATION

The rapid growth of India's gig economy has created a demand for accessible and reliable service platforms. However, existing solutions suffer from several limitations:

- Platforms like Upwork and Fiverr support only digital freelancing and operate primarily in foreign currencies.
- Platforms like Urban Company focus only on field services and restrict direct communication between users.
- There is no unified platform that supports both digital and field services together.
- Lack of trust mechanisms leads to disputes and unreliable service experiences.
- Absence of structured payment systems for milestone-based work.

These challenges highlight the need for a platform that integrates multiple service types, ensures secure transactions, and builds user trust. PocketLancer is motivated by the goal of addressing these gaps and providing a seamless, India-centric solution.

1.2 PROBLEM DEFINITION

The current service marketplace ecosystem in India is highly fragmented and inefficient. Users face difficulty in finding reliable service providers across different domains, while service providers struggle with visibility and fair compensation.

The key problems identified are:

- No unified platform for both digital and field services
- Lack of secure and structured payment mechanisms
- Limited trust and reputation systems
- Absence of real-time communication between users
- Inefficient service discovery without location-based filtering

Due to these limitations, users often rely on multiple platforms or informal networks, leading to inconsistent service quality and trust issues.

1.3 OBJECTIVE OF THE PROJECT

The primary objective of PocketLancer is to design and develop a scalable and reliable marketplace that addresses the limitations of existing platforms.

The specific objectives are:

- To develop a unified platform supporting both digital and field services
- To implement a secure milestone-based payment system using Razorpay
- To enable real-time communication using Socket.IO
- To provide location-based service discovery using geospatial technology
- To establish a trust mechanism through ratings, reviews, and Honor Score
- To ensure seamless user experience across web and mobile platforms

2 LITERATURE SURVEY

Both Mwathi and Shibwabo [1] developed a PHP/Node.js-based collaborative sourcing platform that focused on skill-to-job matching using an event-driven architecture. Their system demonstrated efficient handling of asynchronous workflows, which influenced the design of PocketLancer’s booking and notification pipelines. However, their model did not incorporate geographic proximity as a matching parameter, which is a critical requirement for on-demand field services.

Kaur et al. [2] presented a web-based freelancing portal that supported job posting and bidding mechanisms. While their system addressed basic freelance interactions, it lacked mobile deployment and location-aware search capabilities. These limitations are addressed in PocketLancer through Capacitor-based mobile packaging and MongoDB 2dsphere indexing for geospatial service discovery.

Santy et al. [3] conducted a feasibility analysis of online marketplaces and demonstrated that milestone-based payment contracts significantly reduce disputes by introducing verifiable checkpoints. This concept directly influenced PocketLancer’s implementation of independent milestone escrow payments using Razorpay, ensuring transparency and trust between clients and freelancers.

Batool and Byun [4] proposed a blockchain-based authentication system combined with biometric verification to enhance security in service platforms. Although their approach offers strong cryptographic guarantees, it introduces significant onboarding complexity and friction. In contrast, PocketLancer adopts a lightweight Honor Score mechanism, which serves as a socially interpretable trust metric without requiring complex infrastructure.

Thabassum [5] highlighted that freelancer profiles containing only textual information reduce user confidence and credibility. To address this limitation,

PocketLancer incorporates Cloudinary-based multimedia portfolios, allowing freelancers to showcase images, videos, and project links, along with verified post-service reviews.

Abhinav et al. introduced the CrowdAdvisor system [6], which includes a penalty-based trust model to regulate user behavior. This inspired the design of PocketLancer's Honor Score system, which applies category-aware penalties for actions such as cancellations, disputes, and no-shows, ensuring accountability across both digital and field services.

Madar and Moisi [7] demonstrated that semantic embedding techniques outperform keyword-based matching in handling ambiguous search queries. While PocketLancer currently uses keyword-based search with text indexing, this research suggests a potential future enhancement using AI-driven semantic search for improved accuracy.

Additionally, Resnick et al. [8] established that reputation systems significantly reduce transaction uncertainty in online environments. This reinforces the importance of PocketLancer's integrated review and rating system combined with the Honor Score mechanism.

Bolton et al. [9] further showed that combining escrow systems with reputation mechanisms leads to higher trust and successful transactions. This validates PocketLancer's dual approach of milestone-based escrow payments and cumulative trust scoring.

Lubbers and Albers [10] demonstrated that WebSocket-based communication reduces latency and bandwidth usage compared to traditional HTTP polling. This directly supports PocketLancer's use of Socket.IO for real-time chat and notifications between users.

Aggarwal and Li [11] emphasized the effectiveness of hybrid geospatial systems combining server-side queries and client-side visualization. PocketLancer adopts this

approach using MongoDB geospatial indexing and Leaflet.js for interactive map-based discovery.

Biørn-Hansen et al. [12] analyzed cross-platform mobile development and concluded that web-wrapper technologies provide optimal cost-efficiency for teams with strong web expertise. This supports PocketLancer's use of Capacitor to deliver a consistent experience across web and mobile platforms.

2.1 EXISTING SYSTEM

In the current market, several platforms provide online services for users. These platforms mainly focus on either digital freelancing or field services, but not both.

Digital freelancing platforms such as Upwork, Fiverr, and Freelancer.com allow users to hire professionals for services like web development, graphic design, and content writing. These platforms provide features such as job posting, bidding, and basic communication between users.

On the other hand, platforms like Urban Company provide on-demand field services such as plumbing, electrical work, and home cleaning. These services are usually limited to specific locations and operate based on predefined service categories.

Although these platforms are widely used, they operate independently and do not provide a unified system for all types of services.

2.2 DISADVANTAGES OF EXISTING SYSTEM

The existing systems have several limitations:

- Separate platforms for digital and field services
- Lack of a unified marketplace for all service types
- Limited support for location-based service discovery
- Absence of secure and structured payment mechanisms
- Weak trust systems based only on ratings and reviews
- Limited real-time communication between users
- Restricted flexibility in service selection and interaction

Due to these drawbacks, users often face inconvenience while switching between platforms and may experience trust and payment-related issues.

2.3 PROPOSED SYSTEM

To overcome the limitations of existing systems, the proposed system PocketLancer is developed as a unified service marketplace.

The system integrates both digital freelancing and field services into a single platform. It is developed using the MERN stack, which ensures scalability, flexibility, and efficient data handling.

The proposed system provides the following features:

- Unified platform for both digital and field services
- Real-time communication between users
- Secure payment system with milestone-based transactions
- Location-based service discovery using geospatial technology
- User-friendly interface for both web and mobile platforms
- Trust system using ratings, reviews, and Honor Score

The system is designed to improve user experience and provide a reliable environment for service exchange.

2.4 ADVANTAGES OF PROPOSED SYSTEM

The proposed system offers several advantages over existing systems:

- Combines digital and field services in a single platform
- Provides secure and structured payment mechanisms
- Enables real-time communication between users
- Supports location-based service discovery
- Improves trust through rating and Honor Score system
- Accessible on both web and mobile devices
- Scalable and efficient system architecture

Overall, the proposed system provides a more flexible, reliable, and user-friendly solution compared to existing platforms.

3 SYSTEM ANALYSIS

The analysis phase is a crucial step in the system development process, where the requirements of the system are identified, studied, and documented. It helps in understanding the needs of users and ensures that the system is designed to meet those requirements effectively.

In this project, the analysis focuses on identifying the requirements of a unified service marketplace that supports both digital freelancing and on-demand field services. The system must be scalable, user-friendly, and capable of handling real-time communication, secure transactions, and location-based services.

3.1 SOFTWARE REQUIREMENT SPECIFICATION

The Software Requirement Specification (SRS) defines the functional and non-functional requirements of the system. It serves as a reference for developers, designers, and stakeholders throughout the development lifecycle.

The SRS for PocketLancer includes user requirements, software requirements, and hardware requirements necessary for the proper functioning of the system.

3.1.1 User Requirements

The system is designed to cater to three primary user roles: Clients, Freelancers, and Administrators. Each user type interacts with the system in different ways based on their needs.

Client Requirements

Clients are users who seek services from freelancers. Their requirements include:

- Ability to register and securely log in to the platform
- Search for both digital and field services using filters and categories
- View freelancer profiles, including skills, ratings, and portfolios
- Book services by selecting preferred date, time, and requirements
- Make secure payments using online payment systems
- Communicate with freelancers through real-time chat
- Track booking status and service progress

- Provide feedback, ratings, and reviews after service completion

Freelancer Requirements

Freelancers are service providers who offer their skills on the platform. Their requirements include:

- Ability to register and create a professional profile
- Add details such as skills, experience, pricing, and service category
- Upload portfolio items such as images or project links
- Set availability and manage booking schedules
- Accept or reject service requests from clients
- Communicate with clients through messaging
- Track earnings and completed bookings
- Maintain reputation through ratings and Honor Score

Admin Requirements

Administrators are responsible for managing and monitoring the system. Their requirements include:

- Manage user accounts and verify registrations
- Monitor bookings, payments, and system activities
- Handle disputes between clients and freelancers
- View analytics and performance reports
- Ensure platform security and maintain system integrity

3.1.2 Software Requirements

The system is developed using modern web technologies that ensure performance, scalability, and maintainability.

Frontend Technologies

The frontend of the application is developed using:

- **React.js (Next.js 14)** for building dynamic and responsive user interfaces
- **HTML, CSS, and JavaScript** for structure and styling
- **Tailwind CSS** for efficient and consistent UI design

Next.js provides features such as server-side rendering and client-side rendering, which improve performance and user experience.

Backend Technologies

The backend is developed using:

- **Node.js** for server-side execution
- **Express.js** for building RESTful APIs

The backend handles user authentication, booking management, payment processing, and communication between system components.

Database

- MongoDB is used as the database
- It stores user data, bookings, messages, and other system information
- Its flexible schema design allows efficient handling of different types of data

Additional Tools and Services

To enhance system functionality, the following tools are used:

- **Socket.IO** for real-time communication (chat and notifications)
- **Razorpay** for secure online payments and transaction handling
- **Firebase Cloud Messaging (FCM)** for push notifications
- **Cloudinary** for storing and managing media files
- **Leaflet.js** for map-based service discovery

Operating System

The system can run on multiple operating systems:

- Windows
- macOS
- Linux

Browser Support

The application supports modern web browsers such as:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

3.1.3 Hardware Requirements

The hardware requirements for the system are minimal, as it is a web-based and cloud-hosted application.

- Processor: Minimum Dual-Core Processor
- RAM: 4 GB or higher
- Storage: At least 10 GB free space
- Device: Laptop, desktop, or smartphone
- Internet: Stable internet connection for real-time communication

3.2 SYSTEM ARCHITECTURE DIAGRAM

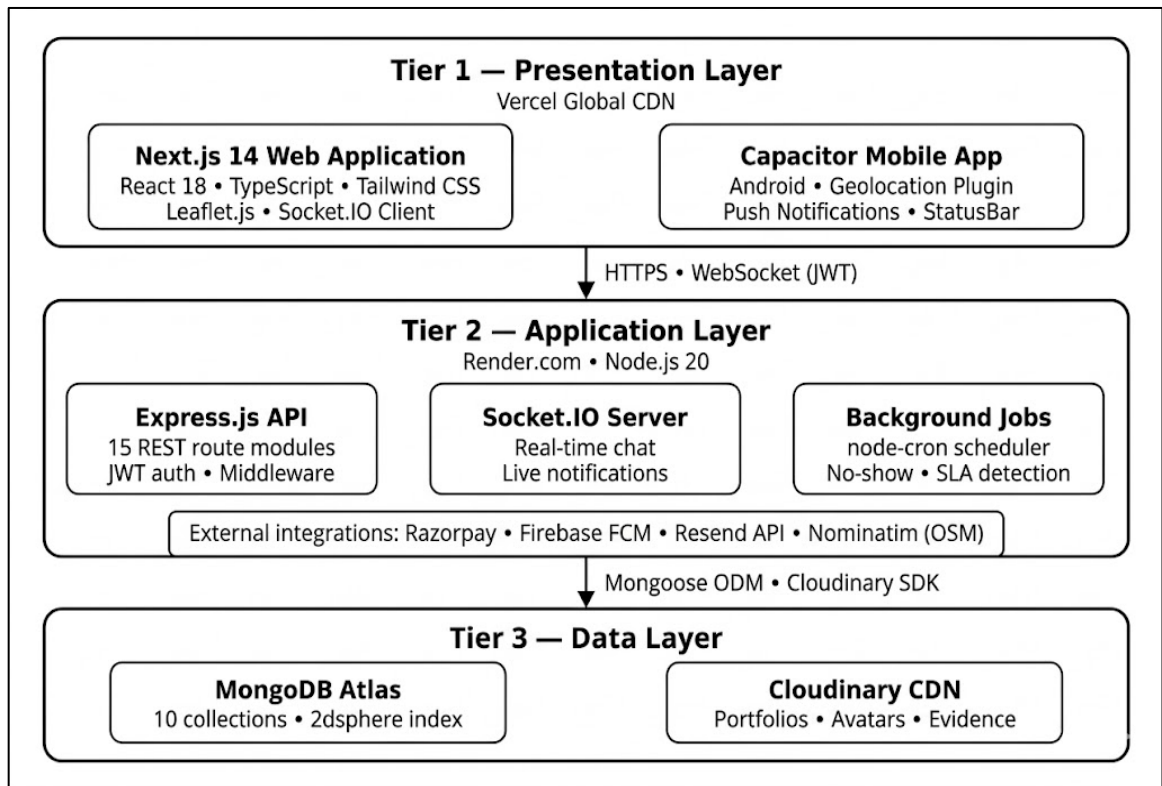


Figure 3.1 PocketLancer Three-Tier Deployment Architecture

The system architecture of PocketLancer represents the overall structure and interaction between different components of the application. The system follows a three-tier architecture, which ensures scalability, modularity, and efficient data management. The three main layers are the Presentation Layer, Application Layer, and Data Layer.

Presentation Layer (Tier 1)

The presentation layer is responsible for user interaction. It includes both the web and mobile interfaces of the system. The web application is developed using **Next.js**, which provides a responsive and dynamic user interface. The mobile application is built using Capacitor, which allows the web application to run as a native Android application.

This layer handles user inputs such as login, service search, booking requests, and communication. It sends requests to the backend through HTTP APIs and receives responses to display data to the user.

Application Layer (Tier 2)

The application layer contains the core business logic of the system. It is developed using Node.js and Express.js, which handle API requests, user authentication, booking management, and payment processing. Additionally, the application layer connects with external services such as:

- Razorpay for secure payment processing
- Firebase Cloud Messaging for push notifications
- Email services for user communication

Data Layer (Tier 3)

The data layer is responsible for storing and managing all system data. MongoDB is used as the primary database to store user information, freelancer profiles, bookings, messages, and reviews.

Media files such as images and portfolio content are stored using Cloudinary, which provides efficient media storage and delivery. This layer ensures data consistency, fast retrieval, and scalability of the system.

4 DESIGN

The design phase focuses on defining the overall architecture, structure, and interaction between different components of the PocketLancer system. It translates the requirements identified during the analysis phase into a structured solution that ensures scalability, maintainability, and efficient system performance.

PocketLancer is designed as a full-stack web and mobile application using a modular and layered architecture. The system integrates multiple technologies to support real-time communication, secure transactions, geolocation-based discovery, and efficient data management. The design ensures that each component operates independently while maintaining seamless interaction with other modules.

The system follows a three-tier architecture consisting of the presentation layer, application layer, and data layer. This separation improves system organization, simplifies development, and allows independent scaling of different components. Additionally, the system incorporates external services such as payment gateways, notification services, and media storage to enhance functionality and user experience.

4.1 INTRODUCTION

The PocketLancer system is designed using the MERN stack, which includes MongoDB, Express.js, React (Next.js), and Node.js. This stack enables a unified JavaScript-based development environment, allowing seamless data flow between the frontend, backend, and database using JSON as the standard data format.

The design adopts a modular approach, where the application is divided into multiple independent modules such as authentication, booking management, payment processing, communication, and notification systems. Each module is responsible for a specific functionality, making the system easier to maintain, extend, and debug.

Scalability is a key consideration in the system design. The use of cloud-based services such as Vercel for frontend deployment and Render for backend hosting ensures that the system can handle increasing user load without performance degradation. MongoDB Atlas provides scalable data storage, while services like Firebase Cloud Messaging and Razorpay enable reliable notification delivery and secure payment processing.

4.2 SYSTEM DESIGN DIAGRAMS

The system diagrams provide a visual representation of the structure, data flow, and operational processes of the PocketLancer platform. These diagrams help in understanding how different components of the system interact with each other and how data moves across various modules.

The diagrams included in this section illustrate the system architecture, database relationships, booking lifecycle, and core workflows such as payment processing and dispute resolution. Together, they provide a comprehensive view of the system design and its functional behavior.

4.2.1 System Architecture Diagram

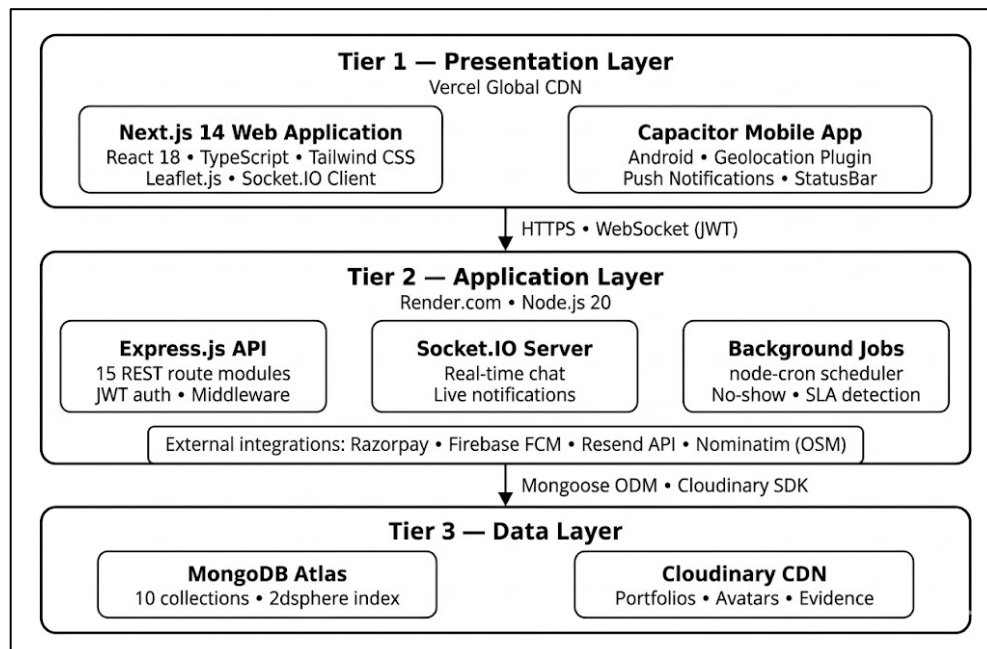


Figure 4.1 System Architecture

The system architecture diagram illustrates the high-level structure of the PocketLancer platform based on a three-tier architecture. It shows how the presentation layer, application layer, and data layer interact with each other to deliver a scalable and efficient system.

The presentation layer includes the web application developed using Next.js and the mobile application built using Capacitor. This layer handles user interactions and communicates with the backend through secure HTTP requests and WebSocket connections.

The application layer consists of the Node.js and Express.js server, which manages business logic, API routing, authentication, and real-time communication using Socket.IO. It also integrates external services such as Razorpay for payment processing, Firebase for push notifications, and email services for user communication.

The data layer includes MongoDB Atlas for storing application data and Cloudinary for managing media files. This layer ensures efficient data storage, retrieval, and scalability.

Overall, the architecture ensures clear separation of concerns, enabling independent development, better maintainability, and improved system performance.

4.2.2 Entity Relationship Diagram

The Entity Relationship (ER) diagram represents the logical structure of the database used in the PocketLancer system and illustrates how different entities are interconnected. Since the system uses MongoDB, a NoSQL database, each entity is implemented as a collection, and relationships are maintained through references and embedded documents where necessary.

The **User** entity forms the foundation of the system, storing essential information such as name, email, role, and Honor Score. Each user can optionally be associated with a **Freelancer** profile, creating a one-to-one relationship. The Freelancer entity contains

additional details such as skills, service category, location, availability, and pricing information, enabling users to offer services on the platform.

The Booking entity is central to the system and represents service interactions between clients and freelancers. It stores information such as service type, booking status, payment details, and milestone data for digital services. A freelancer can have multiple bookings, establishing a one-to-many relationship between Freelancer and Booking. The Review entity is linked to completed bookings and allows clients to provide ratings and feedback. These reviews directly contribute to the freelancer's overall rating and help maintain trust within the platform.

To manage conflicts, the Dispute entity is used to store dispute details, including reason, evidence, and resolution status. For communication, the system includes Conversation and Message entities. A conversation represents a chat session between two users, while messages store individual chat content, enabling real-time interaction.

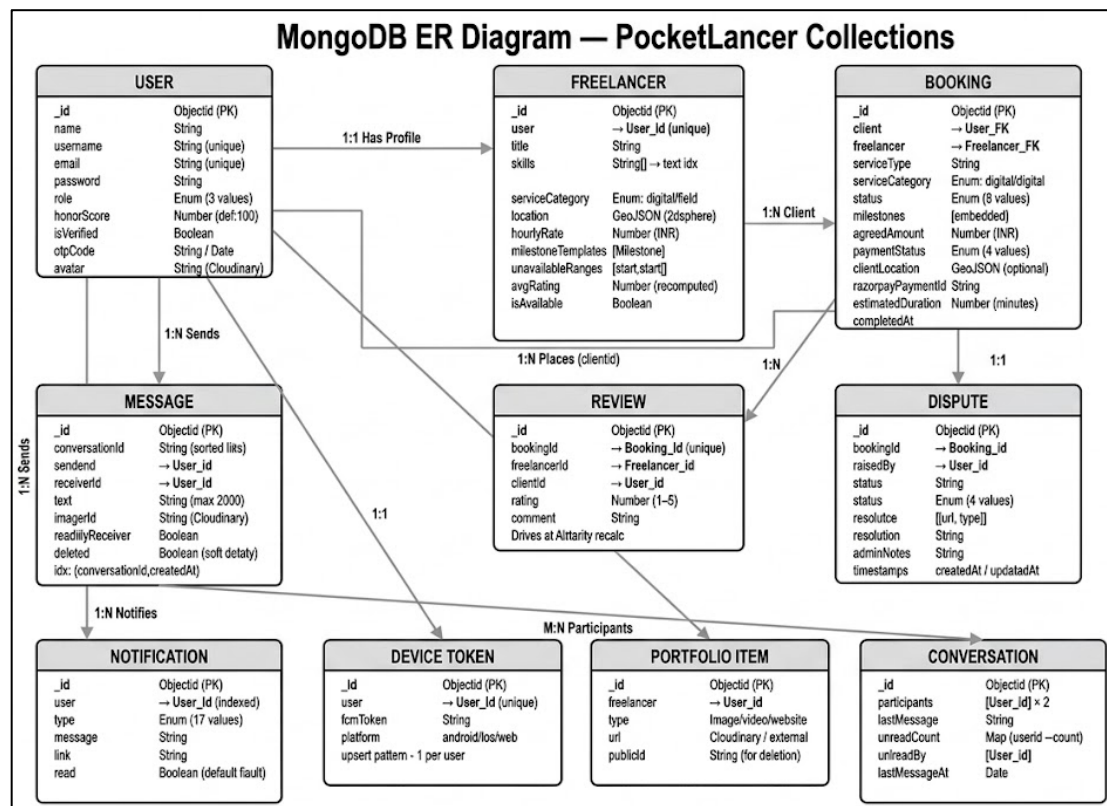


Figure 4.2 Entity Relationship Diagram

4.2.3 Booking State Diagram

The booking state diagram represents the lifecycle of a service request in the PocketLancer system. It shows how a booking transitions through different states based on user actions and system events. Every booking starts in the Pending state when a client places a request. Once the freelancer accepts the request, the booking moves to the Confirmed state. From this stage, the workflow differs slightly depending on whether the service is a field service or a digital service.

For field services, the booking progresses to In Progress when the freelancer begins the service. After completion, it moves to Pending Approval, where the client verifies the work. Upon approval, the booking is marked as Completed.

For digital services, after confirmation, the booking enters an active phase where milestones are executed. Once all milestones are submitted, the booking transitions to Pending Approval, and after client approval, it is marked as Completed.

At any stage, the booking can move to Cancelled or Disputed depending on user actions or system conditions.

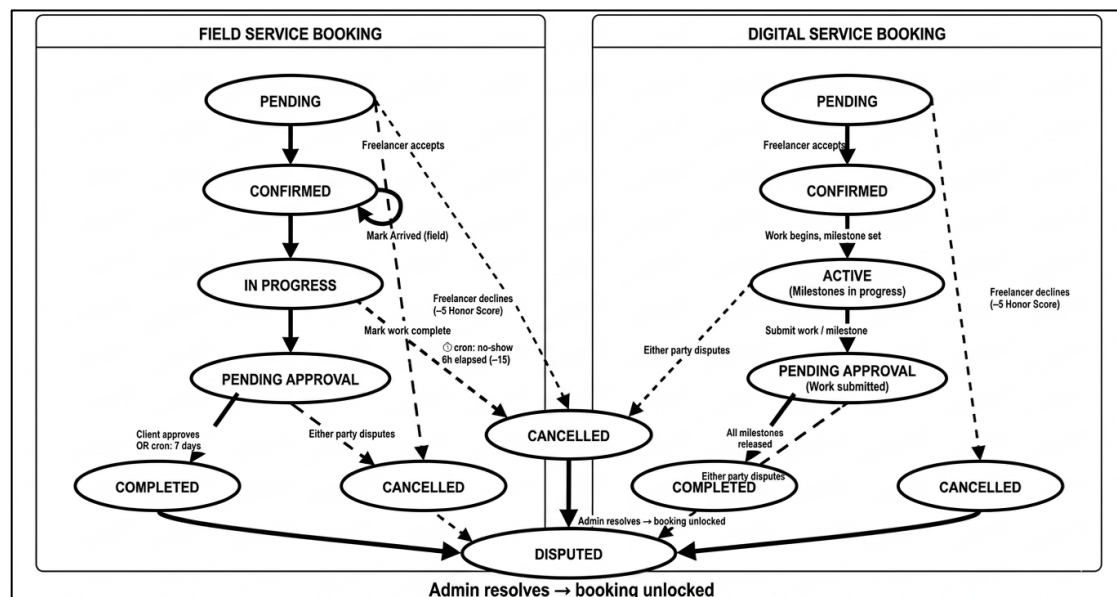


Figure 4.3 Booking Status State Machine Diagram

4.2.4 Payment Workflow Diagram

The payment workflow diagram illustrates the milestone-based escrow payment process used in the PocketLancer platform. It ensures secure and structured transactions between clients and freelancers. In this workflow, the freelancer defines milestone templates, which are associated with a booking when a client initiates a service request. Payments are made by the client for each milestone separately.

When a milestone is completed, the freelancer submits the work for approval. After the client verifies the deliverable, the payment is processed through the Razorpay payment gateway. The system verifies the transaction securely before releasing the payment to the freelancer.

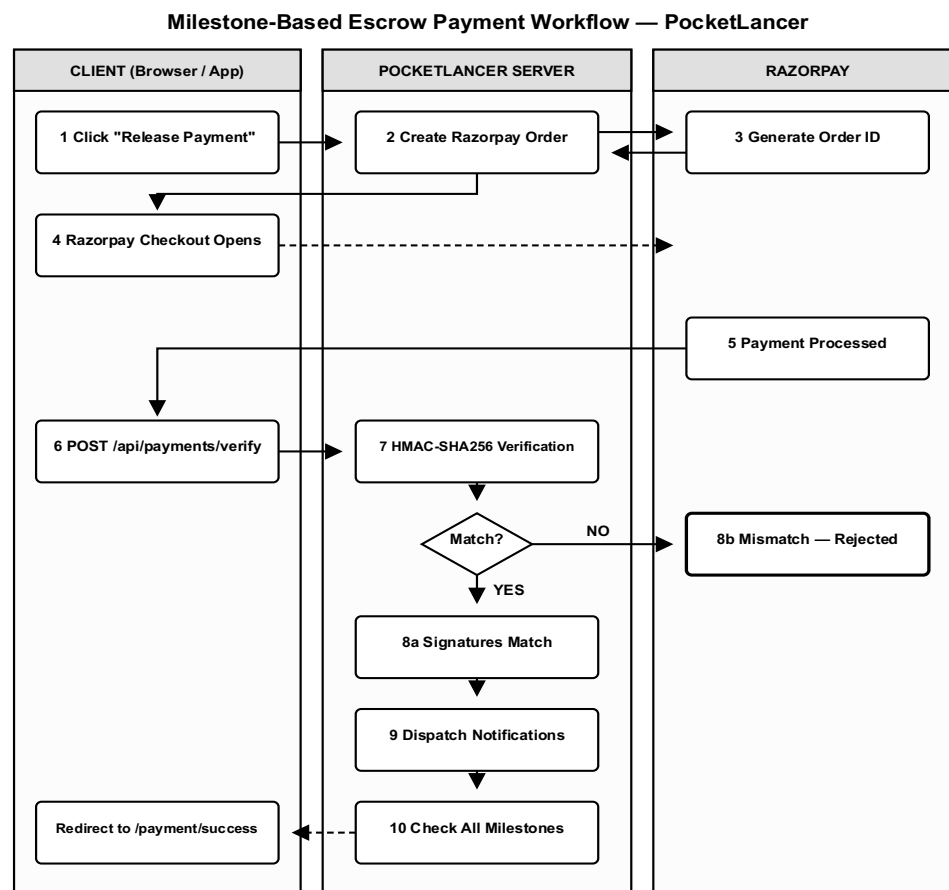


Figure 4.4 Payment Workflow Diagram

4.2.5 Dispute Resolution Diagram

The dispute resolution diagram shows how conflicts between clients and freelancers are handled within the system. It provides a structured mechanism to ensure fairness and transparency.

A dispute can be raised by either party during an active booking. Once a dispute is created, it enters an open state, and both parties can submit supporting evidence. The dispute is then reviewed by the administrator through the admin panel.

Based on the evaluation, the administrator decides the resolution, which may include refunding the client, releasing payment to the freelancer, or applying a shared resolution. The system also updates the Honor Score of the involved users accordingly.

This structured dispute handling mechanism helps maintain trust and ensures proper resolution of conflicts within the platform.

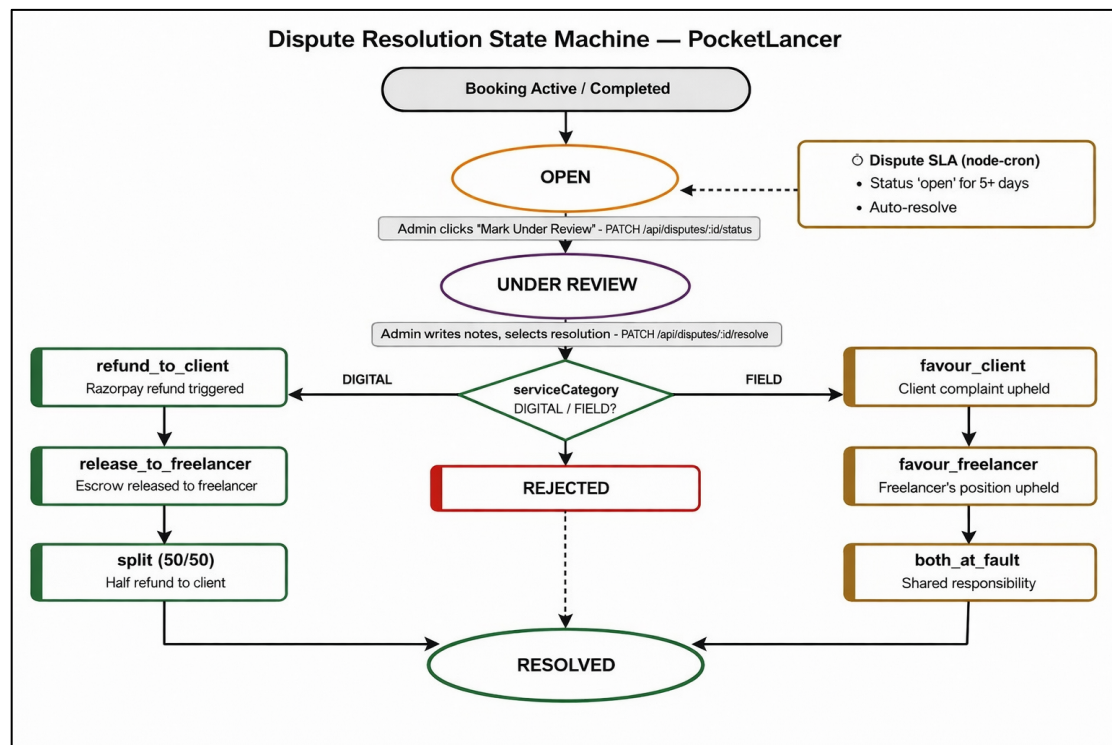


Figure 4.5 Dispute Resolution State Machine Diagram

4.3 DATA DICTIONARY

The data dictionary provides a structured description of the database used in the PocketLancer system. It defines the collections, their fields, data types, and purpose. Since the system uses MongoDB, each entity is represented as a collection, and relationships are maintained using references and embedded documents.

This section helps in understanding how data is organized, stored, and accessed within the system.

4.3.1 User Collection

The User collection stores the basic information of all users in the system, including clients, freelancers, and administrators. It also manages authentication, verification, and trust-related attributes.

Table 4.1 User Collection — Schema Fields

Field	Type	Description
name	String (required)	Full display name
username	String (unique, sparse)	Auto-generated slug (e.g. dedeep-reddy) for /f/[id] SEO routes
email	String (unique)	Login email; normalized to lowercase
password	String (select:false)	bcryptjs hash (never returned in responses)
role	Enum: client/freelancer/admin	Determines routing, permissions, UI

honorScore	Number (default:100)	Cumulative trust metric; floor at 0
otpCode, otpExpiry	String, Date	6-digit OTP + 10-min expiry for email verification
isVerified	Boolean	Activated after OTP verification
passwordResetToken, Expires	String, Date	SHA-256 hash of 30-min reset token
emailNotifications, sms...	Boolean	User notification preferences

4.3.2 Freelancer Collection

The Freelancer collection stores additional information for users who provide services. It includes professional details, service categories, and availability information.

Table 4.2 Freelancer Collection — Schema Fields

Field	Type	Description
user	ObjectId (ref:User)	One-to-one ref; unique constraint
title, skills, bio	String, [String], String	Profile text; compound text index for search
serviceCategory	Enum: digital/field	Determines payment model and state machine
location	GeoJSON Point	2dsphere-indexed [longitude, latitude]
hourlyRate	Number	Base rate in INR

milestoneTemplates	[MilestoneSchema]	Pre-defined milestones; copied into each digital booking
unavailableRanges	[{start,end,note}]	YYYY-MM-DD date ranges; used for booking conflict prevention
avgRating, totalReviews	Number, Number	Recomputed after every new review via aggregation
isAvailable	Boolean	Global availability toggle

4.3.3 Booking Collection

The Booking collection represents service requests between clients and freelancers. It manages booking lifecycle, payments, and scheduling.

Table 4.3 Booking Collection — Schema Fields

Field	Type	Description
clientId, freelancerId	ObjectId refs	Client (→User) and Freelancer document refs
serviceType, serviceCategory	String, Enum	Service name and category (field/digital)
status	Enum (8 values)	Current lifecycle state (see state machine)
milestones	[BookingMilestoneSchema]	Embedded array for digital bookings
agreedAmount, paymentStatus	Number, Enum	Total INR value; pending/partial/completed/refunded

preferredDate, preferredTime	String	Client-requested schedule (field bookings)
clientLocation	GeoJSON Point	Optional client GPS for field bookings
estimatedDurationMinutes	Number	From SERVICE_DURATIONS; used for no-show detection
razorpayOrderId, PaymentId	String	Razorpay identifiers for tracking and refunds

4.3.4 Supporting Collections

The system includes several supporting collections that handle additional functionalities such as communication, push notifications, disputes, freelancer reviews, and digital portfolio management.

Table 4.4 Supporting Collections Summary

Collection	Purpose and Key Fields
Dispute	reason, status (open/under_review/resolved), evidence[], resolution, adminNotes
Review	bookingId (unique), rating 1-5, comment; drives avgRating recalculation
Notification	user, type (17 enum values), message, link, read — persisted in-app history
Message	conversationId, text, imageUrl, readByReceiver — compound index {conversationId, createdAt}
Conversation	participants[], lastMessage, unreadCount (Map), deletedBy[]

DeviceToken	user (unique), fcmToken, platform — one per user for FCM push
PortfolioItem	freelancer, type (image/video/website), url, publicId — Cloudinary-backed

4.3.5 Honor Score System

The Honor Score system is used to maintain trust and accountability among users. It is updated based on user actions and system events.

Table 4.5 Honor Score Events and Penalties

Event	Score Change	Affected User
Dispute (one-sided fault)	-10	Responsible user
Dispute (shared fault)	-5 each	Both users
No-show in field booking	-15	Freelancer
Booking cancellation after acceptance	-5	Freelancer
Successful completion with good rating	+3	Freelancer
Score below threshold	Account flagged/suspended	User

5 IMPLEMENTATION

5.1 METHOD OF IMPLEMENTATION

The PocketLancer system is implemented using the MERN stack, consisting of MongoDB, Express.js, React (Next.js), and Node.js. The system follows a modular architecture where each feature is developed as an independent module, ensuring maintainability and scalability.

The frontend is built using Next.js, providing server-side rendering and optimized performance. The backend is developed using Express.js, exposing RESTful APIs for all functionalities. MongoDB Atlas is used as the database for storing user, booking, and transaction data. Real-time communication is implemented using Socket.IO, while Razorpay is integrated for secure payment processing.

The application is deployed using Vercel for the frontend and Render for the backend, ensuring high availability and scalability.

5.1.1 Express Server & Middleware Chain

The backend of PocketLancer is implemented using Express.js, which serves as the central component responsible for handling API requests, managing application logic, and coordinating communication between different modules.

The server initialization process begins by creating an Express application instance and wrapping it inside an HTTP server. Socket.IO is then attached to this server to enable real-time communication capabilities.

Once the server is initialized, a sequence of middleware functions is applied globally. Middleware plays a critical role in processing incoming requests before they reach the route handlers. The order of middleware execution is important and is carefully structured in this application. First, the CORS (Cross-Origin Resource Sharing) middleware

is applied to allow requests from the frontend application hosted on a different domain. This ensures secure communication between the frontend and backend.

Next, the cookie-parser middleware extracts cookies from incoming requests. This is essential for retrieving JWT tokens stored in HTTP-only cookies for authentication purposes. The Express JSON middleware is then used to parse incoming request bodies in JSON format. This allows the server to easily access and process data sent from the frontend.

Code Implementation

```
// server/index.js — Express server setup
import express from 'express';
import http from 'http';
import cors from 'cors';
import cookieParser from 'cookie-parser';
import mongoose from 'mongoose';
import { Server } from 'socket.io';

const app = express();
const server = http.createServer(app);

// Attach Socket.IO to the same server
const io = new Server(server, {
  cors: {
    origin: process.env.FRONTEND_URL,
    credentials: true,
  },
});

// Global middleware pipeline
app.use(cors({
  origin: process.env.FRONTEND_URL,
  credentials: true,
}));

app.use(cookieParser()); // Parse cookies (JWT)
app.use(express.json({ limit: '10mb' })); // Parse JSON body

// Route modules
app.use('/api/auth', authRoutes);
app.use('/api/profile', profileRoutes);
app.use('/api/freelancers', freelancerRoutes);
```

```

app.use('/api/bookings', bookingRoutes);
app.use('/api/payments', paymentRoutes);

// Connect to MongoDB and start server
mongoose.connect(process.env.MONGODB_URI)
  .then(() => {
    server.listen(process.env.PORT || 5001, () => {
      console.log('Server running successfully');
    });
  })
  .catch(err => console.error('DB connection failed:', err));

```

5.1.2 Authentication & Authorization

The authentication system in PocketLancer is designed to ensure secure access control and user identity verification. It uses JSON Web Tokens (JWT) stored in HTTP-only cookies to maintain user sessions, preventing client-side access to sensitive authentication data. Passwords are securely hashed using bcrypt before storage, ensuring protection against unauthorized access.

When a user registers, their password is hashed and stored in the database. A one-time password (OTP) is generated and sent to the user's email for verification. Once the OTP is validated, the user account is activated and a JWT token is generated.

During login, the system verifies the entered credentials. If valid, a JWT token is issued and stored in an HTTP-only cookie. For subsequent requests, this token is extracted from cookies and verified using middleware. If the token is valid, the user is authenticated and allowed access to protected routes.

Code Implementation

```

// auth utilities
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';

export const hashPassword = async (password) =>
  await bcrypt.hash(password, 12);

export const generateToken = (id, role) =>
  jwt.sign({ id, role }, process.env.JWT_SECRET, { expiresIn: '7d' });

```

```
// protect & authorize middleware
export const protect = async (req, res, next) => {
  const token = req.cookies?.token;
  if (!token) return res.status(401).json({ msg: 'Not authenticated' });

  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.user = await User.findById(decoded.id).select('-password');
  next();
};

export const authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ msg: 'Forbidden' });
  }
  next();
};
```

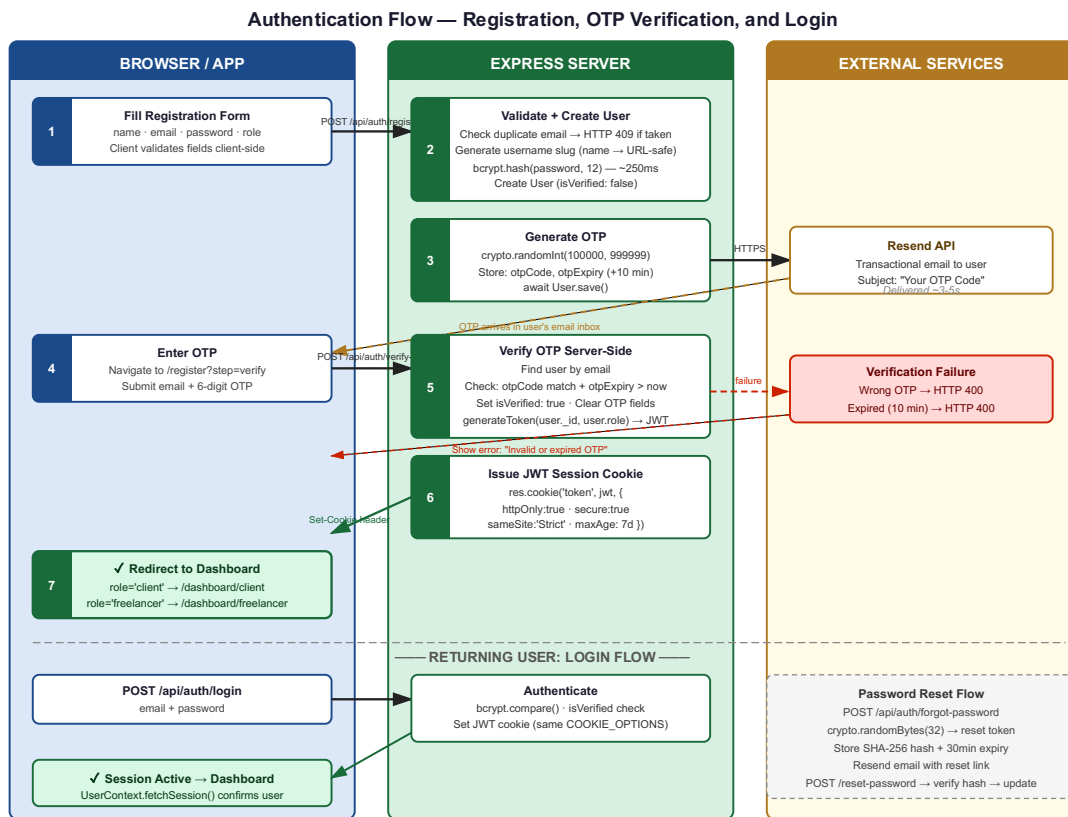


Figure 5.1 Authentication work flow

5.1.3 User & Freelancer Profile Management

The PocketLancer platform provides dedicated profile management features for both clients and freelancers. This module enables users to manage their personal information, while freelancers can additionally define their services, skills, pricing, and availability. The separation of user and freelancer profiles ensures flexibility and supports different roles within the system.

For freelancers, profile creation and updates involve additional fields such as skills, bio, and service configuration. Location data is captured using geolocation APIs and stored in GeoJSON format, enabling location-based search functionality.

Profile image uploads are handled using file upload middleware. The image is processed and stored using a cloud storage service, and the resulting URL is saved in the database. This allows efficient storage and retrieval of media assets.

The system also supports partial updates, meaning users can update only specific fields without affecting the entire profile. This improves usability and reduces unnecessary data processing. The profile management module is designed to be modular and extensible. By separating user and freelancer data, the system maintains clarity in role-specific functionalities. The use of cloud storage for media files reduces server load and improves performance.

Code Implementation

```
// Update user profile
router.patch('/', protect, async (req, res) => {
  const allowedFields = ['name', 'phone', 'address', 'city'];
  const updates = {};

  allowedFields.forEach(field => {
    if (req.body[field] !== undefined) {
      updates[field] = req.body[field];
    }
  });

  const user = await User.findByIdAndUpdate(
    req.user._id,
```

```

    updates,
    { new: true }
  ).select('-password');

  res.json(user);
});
// Update freelancer profile
router.patch('/freelancer', protect, async (req, res) => {
  const freelancer = await Freelancer.findOneAndUpdate(
    { user: req.user._id },
    req.body,
    { new: true, upsert: true }
  );

  res.json(freelancer);
});

```

5.1.4 Search & Geolocation Module

The Search and Geolocation module enables users to efficiently discover freelancers based on skills, services, and location. This module combines keyword-based search with geospatial querying, allowing the platform to support both digital and field service discovery.

For digital services, users can search freelancers using keywords such as skills or service titles. For field services, location plays a critical role, and the system provides a nearby search feature that identifies freelancers within a specified radius from the user's current location. The search functionality operates in two modes: keyword-based search and location-based search.

In keyword-based search, the system uses MongoDB's full-text search capabilities. When a user enters a query, the backend processes it and retrieves matching freelancers based on indexed fields such as title, skills, and bio. This allows efficient filtering even with large datasets.

For geolocation-based search, the user's latitude and longitude are obtained either through browser APIs or mobile device GPS. These coordinates are sent to the backend,

where MongoDB's geospatial query (\$near) is used to find freelancers within a defined radius. The system calculates distance using GeoJSON coordinates stored in the database.

The results are then returned to the frontend, where they can be displayed in both list view and map view. Users can further sort results based on criteria such as distance, rating, or price.

Code Implementation

```
// Geolocation-based nearby search
router.get('/nearby', async (req, res) => {
  const { lat, lng, radius = 10000 } = req.query;

  const freelancers = await Freelancer.find({
    location: {
      $near: {
        $geometry: {
          type: 'Point',
          coordinates: [parseFloat(lng), parseFloat(lat)]
        },
        $maxDistance: parseInt(radius)
      }
    },
    isAvailable: true
  }).limit(50);

  res.json(freelancers);
});

// Keyword-based search
router.get('/search', async (req, res) => {
  const { query } = req.query;

  const results = await Freelancer.find({
    $text: { $search: query }
  }).limit(40);

  res.json(results);
});
```

5.1.5 Booking & Service Management Module

The Booking and Service Management module is the central component of the PocketLancer platform, responsible for connecting clients with freelancers and managing the lifecycle of service requests. It supports both digital services and field services, each with slightly different workflows, while maintaining a unified booking structure.

This module handles the creation, validation, tracking, and status transitions of bookings. It ensures that services are requested, executed, and completed in a structured and reliable manner.

The booking process begins when a client selects a freelancer and submits a booking request. The request includes details such as service type, preferred date and time, and additional notes. For field services, the client can either use their current location or manually enter an address.

Once the request is received, the backend validates the freelancer's availability. It checks whether the selected date conflicts with any unavailable date ranges defined in the freelancer's profile. If a conflict is detected, the booking is rejected; otherwise, it is created with an initial status of *pending*.

After creation, the freelancer can accept or reject the booking. If accepted, the status changes to *confirmed*. For field services, the freelancer can mark arrival, which updates the status to *in progress*. Upon completing the service, the freelancer marks it as completed, moving the booking to *pending approval*. The client then reviews the work and finalizes the booking.

For digital services, the booking may include predefined milestones, which are tracked within the booking document. Each milestone follows its own status progression and is linked to the payment system.

Code Implementation

```
// Socket.IO chat handling
io.on('connection', socket => {
```

```

socket.on('sendMessage', async ({ senderId, receiverId, text }) => {

  // Save message to database
  const message = await Message.create({
    senderId,
    receiverId,
    text
  });

  // Emit message to receiver
  io.to(receiverId).emit('receiveMessage', message);

  // Acknowledge sender
  socket.emit('messageSent', message);
});

// Fetch message history
router.get('/:conversationId/messages', protect, async (req, res) => {
  const messages = await Message.find({
    conversationId: req.params.conversationId
  }).sort({ createdAt: 1 });

  res.json(messages);
});

```

5.1.6 Payment & Escrow Module

The payment system in PocketLancer is designed to ensure secure, transparent, and trust-based transactions between clients and freelancers. Unlike traditional systems where full payment is made upfront or after completion, this platform uses a **milestone-based escrow mechanism**.

In this approach, the total project cost is divided into smaller milestones. Each milestone is paid separately and released only after approval. This reduces financial risk for clients while ensuring freelancers are compensated fairly for completed work.

The payment process begins when a client approves a milestone. The frontend sends a request to the backend to create a Razorpay order. The backend converts the amount into paise (as required by Razorpay) and generates an order ID.

Once the order is created, the Razorpay checkout interface is opened on the client side. The client completes the payment using UPI, card, or net banking. After successful payment, Razorpay returns a payment ID and signature.

The backend then verifies the authenticity of the transaction using HMAC-SHA256 signature verification. If the signature matches, the milestone status is updated to “released,” and both client and freelancer are notified. This process is repeated for each milestone until the entire booking is completed. Several important design decisions were taken while implementing this module:

- **Security:** HMAC verification prevents tampering of payment data.
- **Transparency:** Each milestone is tracked independently.
- **Flexibility:** Supports partial payments instead of full upfront payment.
- **Reliability:** Payment status is stored in the database for recovery and auditing.

The use of Razorpay ensures compatibility with Indian payment systems, especially UPI, which is widely used.

Code Implementation

```
// Step 1: Create Razorpay order
router.post('/order', protect, async (req, res) => {
  const { amount, bookingId } = req.body;

  const order = await razorpay.orders.create({
    amount: Math.round(amount * 100), // INR → paise
    currency: 'INR',
    receipt: bookingId
  });

  res.json({ orderId: order.id });
});

// Step 2: Verify payment signature
router.post('/verify', protect, async (req, res) => {
  const { razorpay_order_id, razorpay_payment_id, razorpay_signature } = req.body;

  const expected = crypto
    .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
```

```

.update(razorpay_order_id + "|" + razorpay_payment_id)
.digest('hex');

if (expected !== razorpay_signature) {
  return res.status(400).json({ msg: 'Verification failed' });
}

res.json({ msg: 'Payment successful' });
});

```

5.1.7 Real-Time Chat System

The Real-Time Chat System in PocketLancer enables seamless communication between clients and freelancers during the service lifecycle. This module plays a crucial role in clarifying requirements, sharing updates, and resolving issues without relying on external communication platforms.

The system is implemented using Socket.IO, which provides bidirectional, event-based communication over WebSockets. This allows messages to be transmitted instantly between users, ensuring real-time interaction within the platform. When a user logs into the system, a Socket.IO connection is established between the client and the server. During this connection, the user is authenticated using a JWT token to ensure that only authorized users can participate in chat communication.

Each conversation is uniquely identified using a conversation ID, which is generated based on the participating users. This ensures that messages between the same pair of users are grouped consistently, regardless of who initiates the conversation.

When a user sends a message, it is first stored in the database to ensure persistence. The server then emits the message to the recipient using Socket.IO, allowing it to appear instantly in the chat interface. If the recipient is offline, the message remains stored and is retrieved when they reconnect.

The system also supports additional features such as image sharing and unread message tracking, enhancing the overall communication experience.

The chat system is designed to balance real-time performance with data reliability. By storing messages in the database, the system ensures that conversations are not lost even if users disconnect.

Using Socket.IO allows efficient event-driven communication, reducing latency compared to traditional polling methods. Authentication at the socket level prevents unauthorized access to chat functionality.

Code Implementation

```
// Socket.IO chat handling
io.on('connection', socket => {

  socket.on('sendMessage', async ({ senderId, receiverId, text }) => {

    // Save message to database
    const message = await Message.create({
      senderId,
      receiverId,
      text
    });

    // Emit message to receiver
    io.to(receiverId).emit('receiveMessage', message);

    // Acknowledge sender
    socket.emit('messageSent', message);
  });

  // Fetch message history
  router.get('/:conversationId/messages', protect, async (req, res) => {
    const messages = await Message.find({
      conversationId: req.params.conversationId
    }).sort({ createdAt: 1 });

    res.json(messages);
  });
});
```

5.1.8 Notification & Honor Score System

The Notification and Honor Score system in PocketLancer is designed to enhance user engagement, transparency, and trust within the platform. While the notification system

ensures that users are informed about important events in real time, the Honor Score system acts as a reputation mechanism that reflects user reliability and behavior.

Together, these two components create a feedback-driven ecosystem where users are continuously informed and held accountable for their actions. The notification system operates in two modes: real-time delivery and persistent storage. When an event occurs—such as booking creation, payment completion, or dispute resolution—a notification is generated on the server.

This notification is stored in the database to maintain a history of user activities. At the same time, it is delivered instantly to the user using Socket.IO if the user is online. For mobile users, push notifications can also be triggered using external services.

On the other hand, the Honor Score system maintains a numerical score for each user, typically starting at a default value. This score is adjusted based on user actions. Positive actions such as successful booking completion and good reviews may increase the score, while negative actions such as cancellations, no-shows, or disputes result in score deductions.

The system ensures that the score remains within defined limits (e.g., 0 to 100) and triggers alerts or restrictions when thresholds are crossed.

The notification system is designed to be event-driven, ensuring that users receive updates without needing to manually refresh the application. Storing notifications in the database ensures persistence, allowing users to review past activities.

The Honor Score system is implemented as a simple yet effective trust metric. Instead of complex algorithms, a rule-based approach is used to maintain transparency and ease of understanding. This makes the system predictable and fair for users.

Code Implementation

```
// Notification utility
export async function notify(userId, type, message) {
```

```

// Save notification in database
await Notification.create({
  user: userId,
  type,
  message
});

// Send real-time notification
if (io) {
  io.to(userId.toString()).emit('notification', {
    type,
    message,
    createdAt: new Date()
  });
}
}
// Honor score update
export async function updateHonorScore(userId, change) {
  await User.findByIdAndUpdate(userId, {
    $inc: { honorScore: change }
  });
}

```

5.1.9 Dispute Resolution System

The Dispute Resolution System in PocketLancer is designed to handle conflicts between clients and freelancers in a structured and transparent manner. Since the platform involves financial transactions and service delivery, disputes may arise due to delays, incomplete work, or dissatisfaction. This module ensures that such issues are resolved fairly with proper evidence and administrative intervention. The system allows both parties to raise disputes during an active booking, ensuring that conflicts are addressed before final completion or payment release.

A dispute can be initiated by either the client or the freelancer through the booking interface. When a dispute is raised, the booking is temporarily locked to prevent further actions such as payment release or status changes.

The user is required to provide a reason along with supporting evidence, which may include images or documents. This information is stored in the database and associated with the corresponding booking.

Once a dispute is created, it enters an *open* state and becomes visible to the admin panel. The administrator reviews the dispute details, examines the submitted evidence, and analyzes the booking history.

Based on the evaluation, the admin selects an appropriate resolution. Possible outcomes include refunding the client, releasing payment to the freelancer, or applying a shared resolution where both parties bear partial responsibility

After resolution, the dispute status is updated, the booking is unlocked, and necessary actions such as payment refunds or Honor Score adjustments are applied. The dispute system is designed to ensure fairness, transparency, and accountability. Locking the booking during an active dispute prevents misuse or premature actions.

Code Implementation

```
// Create dispute
router.post('/disputes', protect, async (req, res) => {
  const { bookingId, reason } = req.body;

  const dispute = await Dispute.create({
    bookingId,
    raisedBy: req.user._id,
    reason,
    status: 'open'
  });

  res.status(201).json(dispute);
});

// Resolve dispute (admin)
router.patch('/:id/resolve', protect, async (req, res) => {
  const { resolution } = req.body;

  const dispute = await Dispute.findByIdAndUpdate(
    req.params.id,
    { status: 'resolved', resolution },
    { new: true }
  );

  res.json(dispute);
});
```

5.1.10 Admin Dashboard & Analytics

The Admin Dashboard and Analytics module provides centralized control and monitoring capabilities for the PocketLancer platform. It allows administrators to oversee system activities, manage users, handle disputes, and analyze platform performance through key metrics.

When the admin accesses the dashboard, the frontend sends requests to the backend to retrieve system statistics. These include total number of users, freelancers, bookings, and disputes. The backend processes these requests by querying multiple collections and aggregating data.

The admin module is designed with a focus on performance and scalability. Using concurrent database queries reduces response time when fetching multiple metrics. Role-based access control ensures that sensitive administrative functionalities are protected. The dashboard presents summarized data instead of raw records, improving readability and usability.

The system also supports filtering and searching of users, enabling efficient management even as the platform grows. By integrating analytics with operational controls, the module provides a comprehensive view of the system.

Code Implementation

```
// Fetch admin statistics
router.get('/stats', protect, async (req, res) => {

  const [users, freelancers, bookings, disputes] = await Promise.all([
    User.countDocuments(),
    Freelancer.countDocuments(),
    Booking.countDocuments(),
    Dispute.countDocuments()
  ]);

  res.json({
    users,
    freelancers,
    bookings,
```

```

    disputes
  });
});
// Get users list
router.get('/users', protect, async (req, res) => {

  const users = await User.find()
    .select('-password')
    .limit(100)
    .sort({ createdAt: -1 });

  res.json(users);
});

```

5.2 SYSTEM OUTPUT AND SCREENSHOTS

The Output Screens section presents the visual representation of the PocketLancer application. The following screens highlight the major functionalities of the system, including authentication, search, booking, communication, and administration.

5.2.1 Landing Page and Registration

The landing page renders with Framer Motion animations and correctly adapts the primary CTA button based on authentication state.

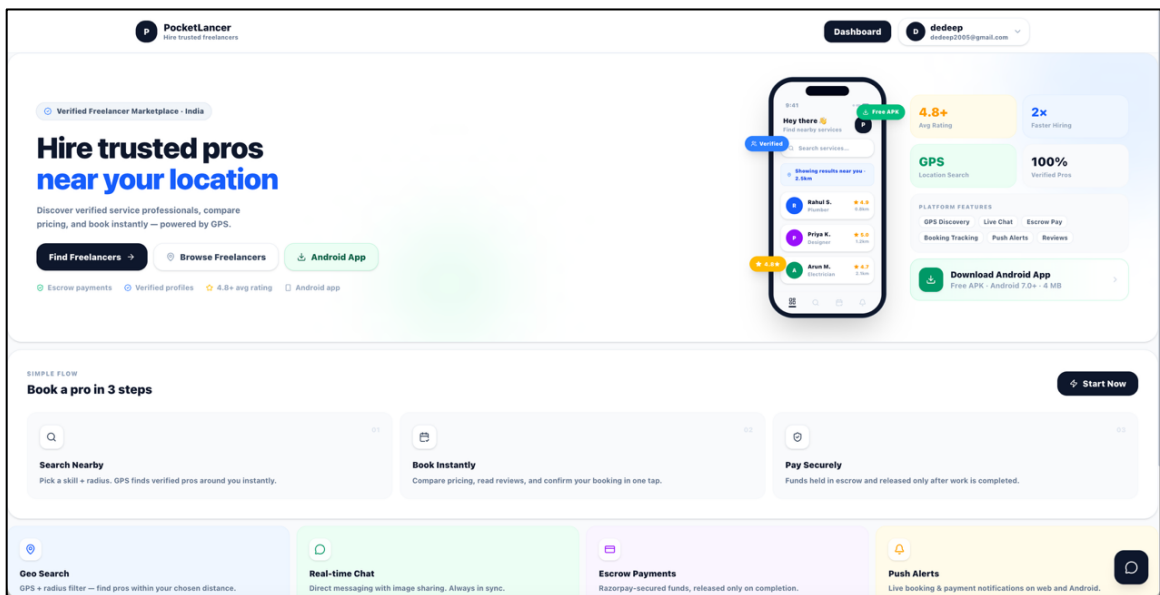


Figure 5.2 PocketLancer Landing Page

5.2.2 Dashboards

The client dashboard correctly aggregates booking counts, renders the Honor Score bar with correct colour-coding (green ≥ 75 , orange 35-74, red < 35), and the 'Find Services Near Me' shortcut calls `getCurrentLocation()` and navigates to `/map-search` with coordinates. The freelancer dashboard's Today's Field Jobs widget filters confirmed field bookings for the current date. an FCM notification to the client within 2-3 seconds.

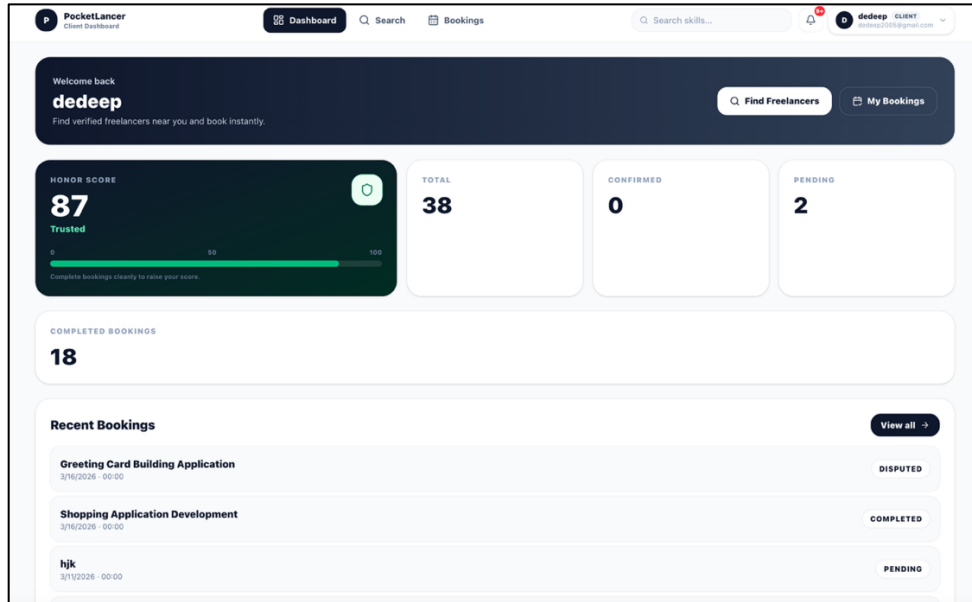


Figure 5.3 Client Dashboard with Honor Score Indicator

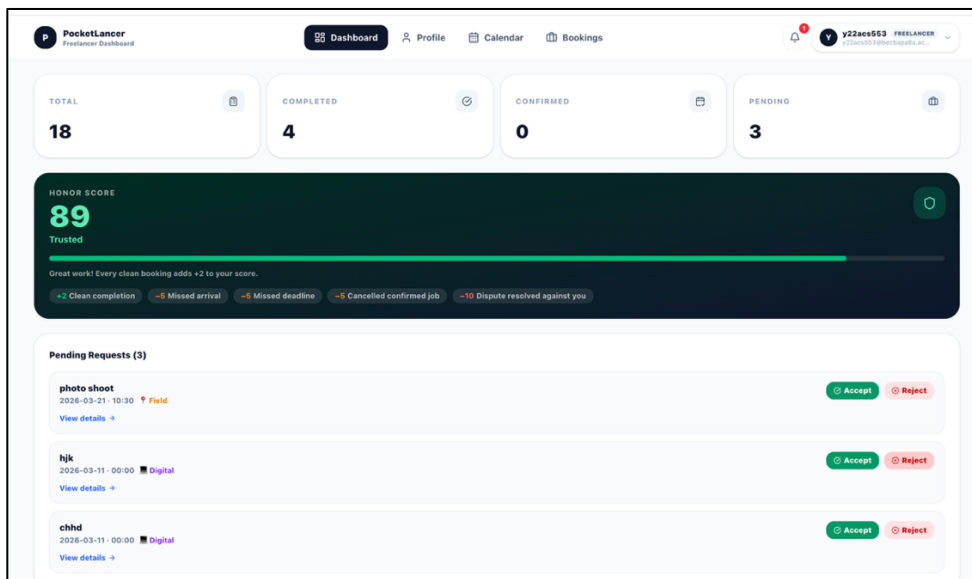


Figure 5.4 Freelancer Dashboard — Today's Field Jobs Widget

5.2.3 Search and Map Discovery

Keyword search returns relevant results for 'plumber', 'react developer', and 'AC technician' queries. Search variant expansion improves recall for morphological variants. Geospatial map search places markers accurately at freelancer-stored coordinate.

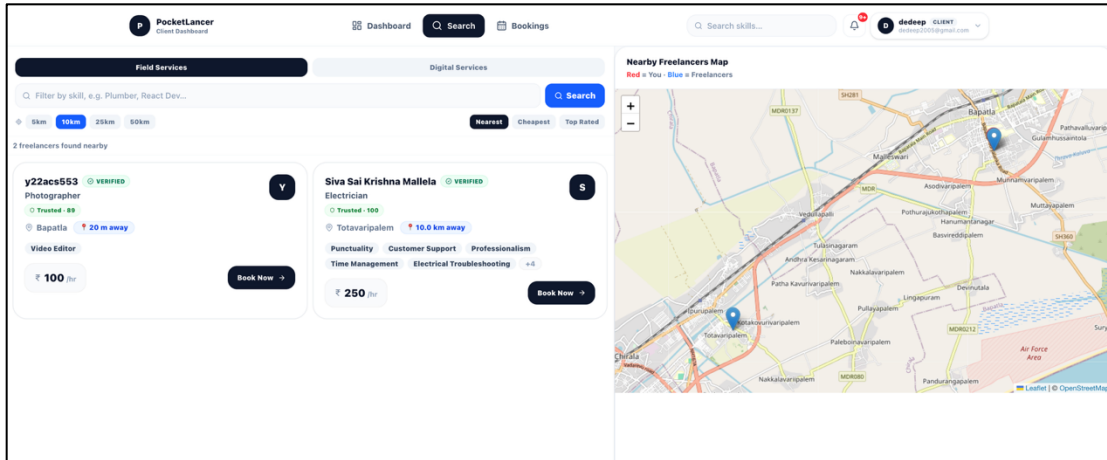


Figure 5.5 Search Results and Map-Based Discovery

5.2.4 Booking Creation Interface

The booking screen allows users to initiate service requests by selecting preferred dates, service types, and location details.

The screenshot shows the 'Confirm Booking' screen in the PocketLancer client dashboard. At the top, there's a navigation bar with 'Dashboard', 'Search', and 'Bookings' tabs. Below this, a search bar is present. The main content area is titled 'Confirm Booking' and includes a note: 'Your payment is held in escrow — released only after you approve the work.' The form consists of several sections: 'Service Type' with a dropdown menu showing 'Thumbnail Design for Youtube Video'; 'Description (optional)' with a text area containing 'Complete information will be sent through chat window'; 'How long will this take?' with a section for selecting the duration. The duration is currently set to '8 hours'. At the bottom, there's a large purple button labeled 'Pay ₹200 & Confirm'.

Figure 5.6 Service Booking Form

5.2.5 Booking and Milestone Payment

The digital booking lifecycle was tested end-to-end in Razorpay's test environment. Milestone state transitions enforced correctly: approving a non-submitted milestone returns HTTP 400.

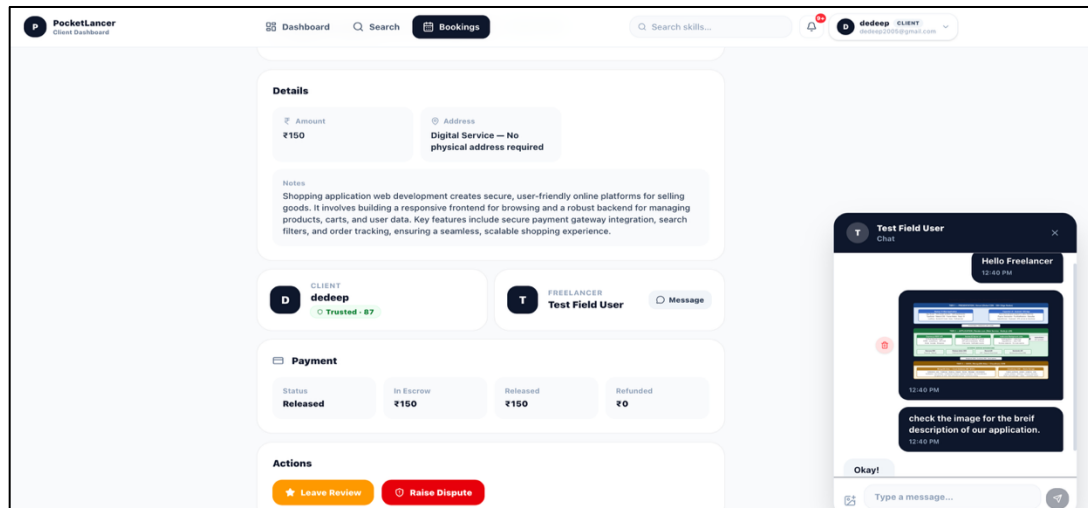


Figure 5.7 Booking Detail — Milestone Payment Flow

5.2.6 Admin Dispute Resolution

Dispute filing with JPEG evidence upload, admin status update, resolution selection, Honor Score penalty application (−10), and Razorpay refund API call all tested successfully.

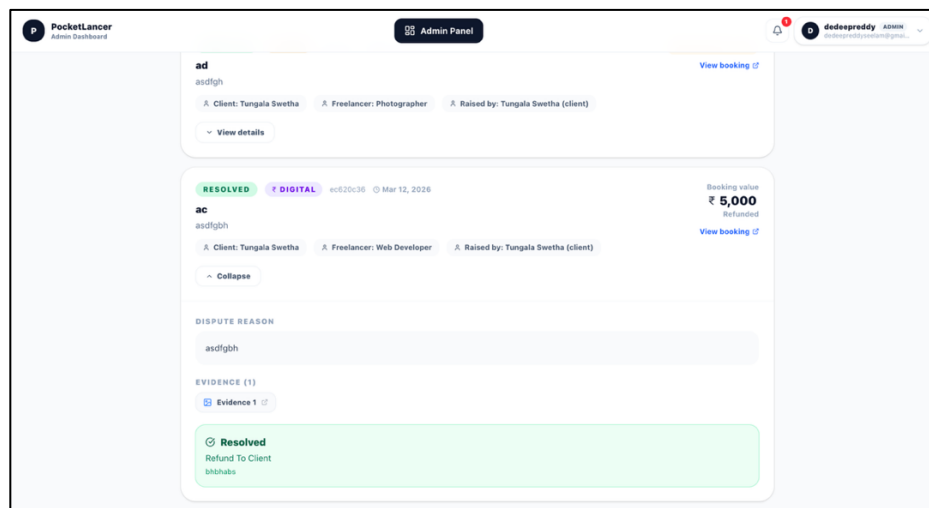


Figure 5.8 Admin Dispute Resolution

5.2.7 Chat and Communication Interface

The chat interface enables real-time communication between clients and freelancers using WebSocket technology.

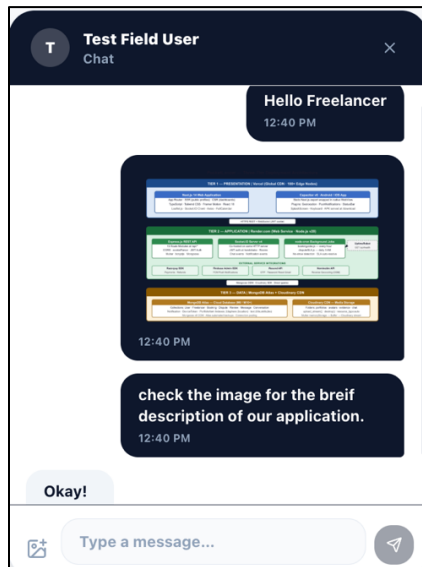


Figure 5.9 Chat Window

5.2.8 Notification System Interface

This screen showcases the notification center, where users receive updates related to bookings, payments, disputes, and system alerts. Notifications are delivered both in-app and via push notifications, ensuring timely communication. The interface allows users to mark notifications as read and navigate to relevant sections.

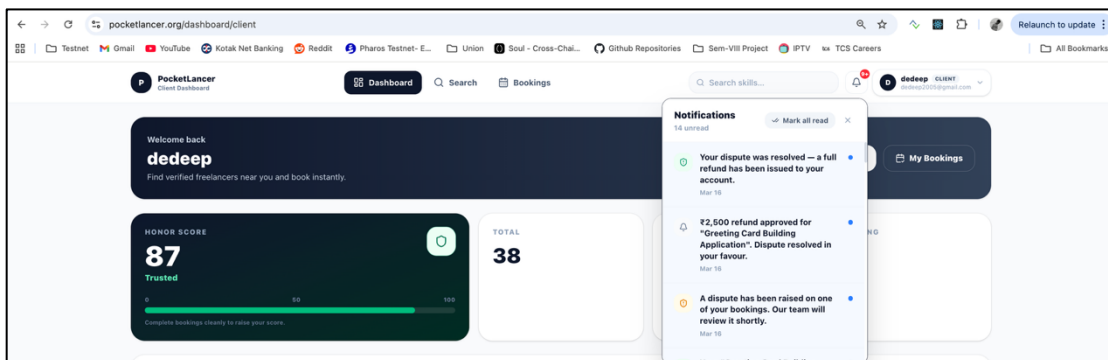


Figure 5.10 Notification Panel

5.2.9 Freelancer Profile Editing Interface

The freelancer profile editing screen allows users to manage and update their professional information in a structured and user-friendly interface. It includes sections for profile completion, identity details, service category selection (field or digital), pricing, skills, and portfolio inputs.

The screenshot displays the 'Freelancer Profile' editing interface. At the top, there's a navigation bar with 'Dashboard', 'Profile', 'Calendar', and 'Bookings'. The 'Profile' section is active, showing 'Edit Your Profile' with buttons for 'Preview Public Profile', 'Visible to clients', and 'Save Profile'. Below this, a 'Your Honor Score' of 89/100 is shown. The 'Profile Completion' section indicates 40% completion with a progress bar and a list of items to complete: Profile photo, Professional title, Bio (30+ chars), 5+ skills, Location, and 2+ past works. The 'Your Rating' section shows a 3/5.0 rating with 1 review received and a rate of ₹100/hr. The 'Identity & Category' section includes fields for 'Professional Title' (Photographer) and 'Service Category' (Field Service). A 'Reviews' section shows a review from 'dedeep' with a 3-star rating. The interface is clean and modern, with a sidebar on the left and a top navigation bar.

Figure 5.11(a) Freelancer Profile Editing Screen

The screenshot displays the 'Freelancer Profile' editing interface, focusing on the 'Service Category' and 'Identity' sections. The 'Service Category' section shows 'Field Service' selected. The 'Identity' section includes fields for 'Full Name' (Dedeep), 'Phone Number' (9581030428), and 'Date of Birth' (2005-02-12). The 'Bio' section has a text area for 'Describe your expertise, years of experience, approach to work, and why clients should choose you...'. The 'Pricing' section has a 'Hourly Rate (₹/hr)' field with '100'. The interface is clean and modern, with a sidebar on the left and a top navigation bar.

Figure 5.11(b) Freelancer Profile Editing Screen

5.2.10 Portfolio Management Interface

This screen allows freelancers to upload and manage their work samples, including images, videos, and external links.

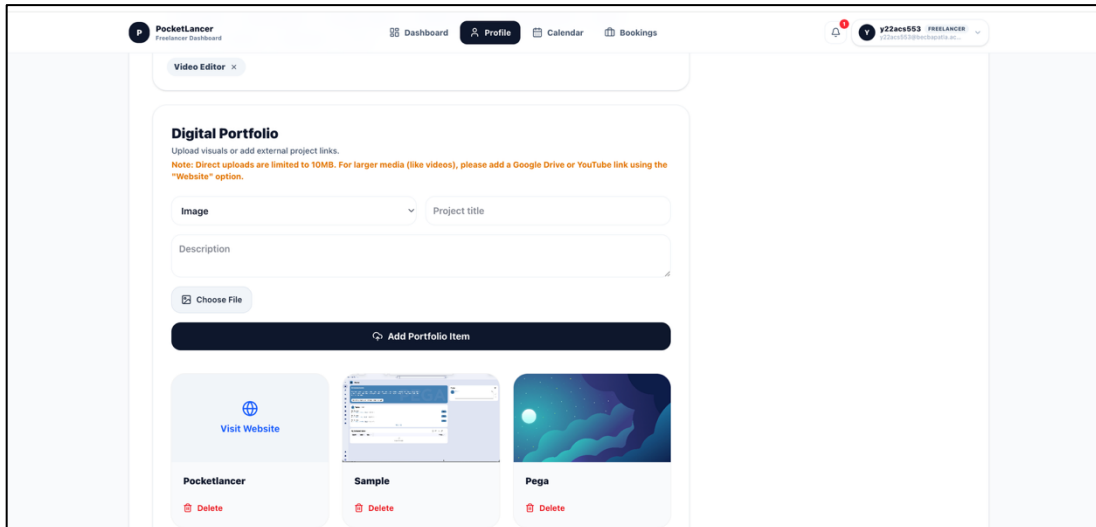


Figure 5.12 Digital Portfolio for Freelancers

5.2.11 Admin Dashboard

The admin dashboard provides system-wide insights, including user statistics, booking metrics, and dispute records. It allows administrators to monitor platform activity and make data-driven decisions.

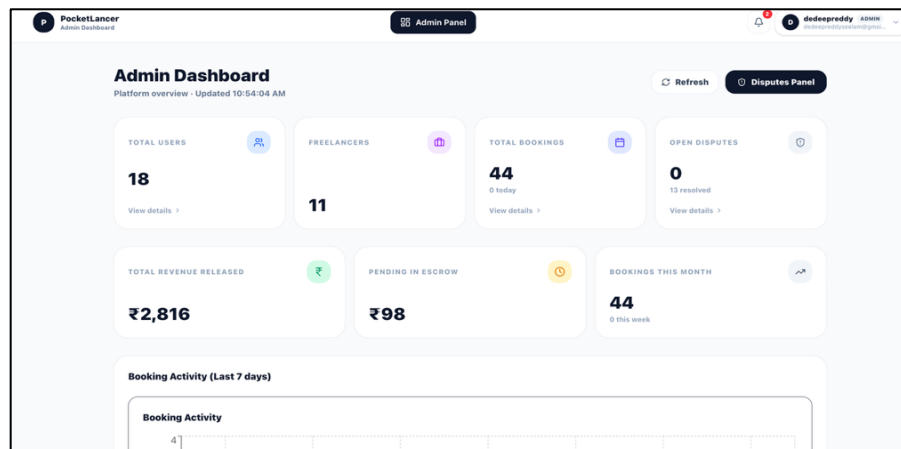


Figure 5.13 Admin Dashboard

5.2.12 Dispute Resolution Panel

This screen illustrates the admin dispute management interface, where disputes can be reviewed and resolved.

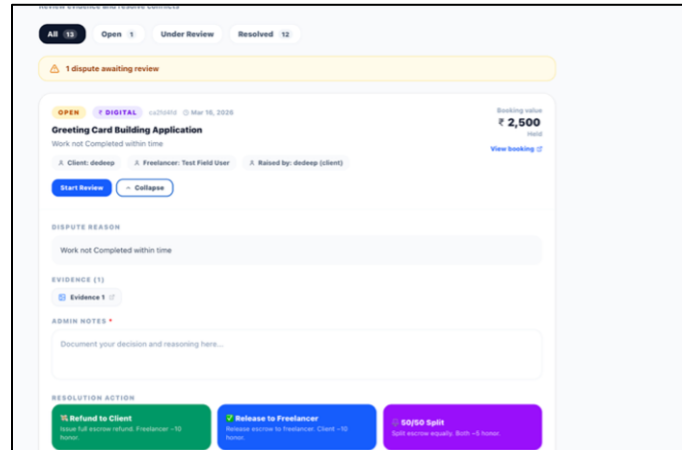


Figure 5.14 Admin Dispute Resolution Panel

5.2.13 Mobile Application Screens

The mobile application screens demonstrate the responsiveness and adaptability of the PocketLancer platform on Android devices using Capacitor.

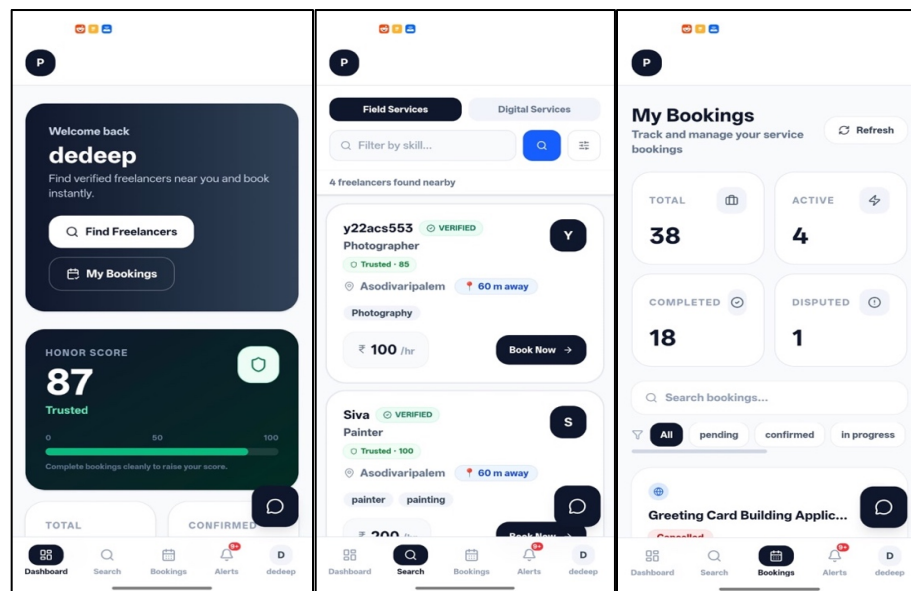


Figure 5.15 Mobile Application Interface

5.3 RESULTS AND ANALYSIS

The performance evaluation of the PocketLancer system indicates that the application meets the expected efficiency and responsiveness requirements across all major modules. The REST API layer demonstrates consistently low response times, ensuring smooth communication between the frontend and backend. This reflects the effectiveness of the Express.js architecture, optimized middleware pipeline, and efficient database querying strategies implemented in the system.

The database performance, particularly for geospatial and full-text search operations, shows significant optimization due to the use of MongoDB indexing techniques such as 2dsphere and compound text indexes. These optimizations enable rapid retrieval of nearby freelancers and accurate search results, thereby enhancing user experience in service discovery. The observed performance confirms that the system can handle location-based queries efficiently even under real-time conditions.

Real-time communication and notification mechanisms also perform effectively within acceptable latency ranges. The integration of Socket.IO ensures near-instant message delivery, enabling continuous interaction between clients and freelancers without noticeable delays. Messages are transmitted and received in real time, even during active booking sessions, which improves coordination and reduces dependency on external communication tools. Additionally, the system maintains message persistence, ensuring that conversations are not lost during temporary disconnections.

The push notification system using Firebase Cloud Messaging reliably delivers alerts related to bookings, payments, and system updates across devices. Notifications are triggered instantly and reach users even when the application is running in the background. This ensures that users remain informed about important events without needing to actively monitor the platform. Together, these features significantly enhance user engagement, improve communication efficiency, and contribute to the overall responsiveness and reliability of the system.

On the frontend side, the application achieves good rendering performance and user experience, supported by optimized Next.js deployment on a global CDN. The system loads quickly and maintains consistent performance across devices, including mobile environments powered by Capacitor. Overall, the results confirm that the system is scalable, reliable, and capable of delivering a responsive user experience across all core functionalities.

Table 5.1 API and System Performance Benchmarks

Metric	Target	Observed
REST API average response time	< 300 ms	~175 ms
MongoDB 2dsphere \$near query	< 200 ms	~85 ms
Socket.IO message latency (LAN)	< 100 ms	~50 ms
Cloudinary image upload (< 1 MB)	< 3 s	~1.4 s
Razorpay Checkout widget load	< 2 s	~1.3 s
FCM push delivery (Android)	< 5 s	~2.1 s
Next.js First Contentful Paint (Vercel)	< 2.5 s	~1.8 s
OTP email delivery (Resend API)	< 10 s	~3.5 s
MongoDB full-text search	< 250 ms	~120 ms
Capacitor SplashScreen duration	1.2 s	1.2 s (configured)

6 TESTING & VALIDATION

Testing and validation are critical phases in the development of the PocketLancer system to ensure that the application functions correctly, securely, and efficiently under various conditions. The system was tested across multiple modules, including authentication, booking, payment processing, real-time communication, and admin controls. Both functional and non-functional testing approaches were adopted to verify correctness, performance, and reliability.

The testing process involved validating user inputs, API responses, database operations, and system workflows. Special attention was given to edge cases such as invalid inputs, unauthorized access, and failure scenarios. The results confirm that the system behaves as expected and meets the defined requirements.

6.1 FEATURES TO BE TESTED

The following core features of the PocketLancer system were identified and thoroughly tested:

6.1.1 User Authentication and Authorization

The authentication module was tested for registration, login, OTP verification, and password reset functionality. Validation checks ensured that duplicate email registrations are prevented, expired OTPs are rejected, and secure session handling is maintained.

6.1.2 Freelancer and Profile Management

Profile creation and editing functionalities were tested to ensure that users can update their details without errors. Validation of required fields, file uploads (profile image), and real-time updates were verified.

6.1.3 Search and Geolocation

The search functionality was tested using different keyword inputs to verify relevance and accuracy of results. Geolocation features were validated to ensure correct detection of user location and proper filtering of nearby freelancers. Edge cases such as invalid coordinates were also tested.

6.1.4 Booking and Scheduling System

The booking module was tested for both field and digital services. The system correctly prevents booking on unavailable dates and ensures proper state transitions across booking stages. Conflict detection and scheduling validations were verified to ensure reliability.

6.1.5 Payment Processing System

The Razorpay integration was tested for order creation, payment processing, and verification. Invalid payment signatures were rejected successfully, ensuring system security. Milestone-based payment flow was validated for correctness and reliability.

6.1.6 Real-Time Communication

The chat system using Socket.IO was tested for message delivery, reconnection handling, and synchronization. Messages were successfully delivered even after temporary network interruptions, confirming system robustness.

6.1.7 Notification System

Push notifications and in-app notifications were tested for different events such as booking updates, payments, and disputes. Notifications were delivered reliably across devices, including background Android applications.

6.1.8 Admin Controls and Security

Admin functionalities such as dispute resolution, user management, and analytics were tested. Unauthorized access attempts were blocked successfully, ensuring secure system operation.

6.1.9 Background Jobs and Automation

Automated processes such as no-show detection and dispute SLA resolution were tested using scheduled cron jobs. These processes executed correctly based on defined conditions and timelines.

6.2 FEATURES NOT TO BE TESTED

Certain aspects of the system were excluded from detailed testing due to scope limitations or dependency on external services:

6.2.1 Third-Party Service Internals

The internal functioning of external services such as Razorpay, Cloudfire, Firebase Cloud Messaging, and Resend API was not tested. Only their integration and response handling within the system were validated.

6.2.2 Network Infrastructure

Variations in network latency, bandwidth limitations, and server-side infrastructure outside the application scope were not deeply tested. However, basic responsiveness under normal conditions was verified.

6.2.3 Device-Specific Hardware Behavior

Hardware-specific behaviors such as GPS accuracy variations, device battery performance, and OS-level optimizations were not extensively tested, as they depend on the user's device environment.

6.2.4 Large-Scale Load Testing

Although functional testing was performed thoroughly, extensive stress testing with thousands of concurrent users was not conducted due to resource constraints.

6.3 TESTING TOOLS AND ENVIRONMENT

The testing of the PocketLancer system was carried out using a combination of manual testing and development tools in a controlled environment.

Development Environment

- Frontend: Next.js (React-based framework)
- Backend: Node.js with Express.js
- Database: MongoDB Atlas (Cloud database)
- Mobile Platform: Capacitor (Android)

Testing Tools

- **Postman:** Used for testing REST API endpoints, validating request-response cycles, and checking error handling.
- **MongoDB Compass:** Used for verifying database records and ensuring correct data storage.
- **Browser Developer Tools:** Used for debugging frontend components, monitoring network requests, and analyzing performance.
- **Razorpay Test Mode:** Used to simulate payment transactions safely.
- **Firebase Console:** Used to test push notifications.
- **Console Logs and Debugging Tools:** Used for runtime debugging and validation.

Execution Environment

The system was tested on:

- Desktop browsers (Chrome)
- Android mobile devices (via Capacitor app)

- Stable internet connection

The testing environment ensured that all modules function correctly across different platforms and devices.

6.4 DESIGN OF TEST CASES AND SCENARIOS

The system was tested using predefined test cases covering both normal and edge scenarios. These test cases were designed to validate system behavior under different conditions and ensure robustness.

The results of the test scenarios indicate that all major functionalities of the system are working as expected. Input validation mechanisms successfully prevent invalid operations, and security checks ensure protection against unauthorized access. The system correctly handles edge cases such as duplicate entries, invalid inputs, and failed transactions.

Automated processes such as booking cancellation and dispute resolution were also validated through scheduled job execution. Real-time features such as chat and notifications performed reliably under different conditions, including network interruptions.

Overall, the testing results confirm that the PocketLancer system is stable, secure, and ready for deployment. The successful execution of all test scenarios demonstrates that the system meets its functional requirements and provides a reliable user experience.

Table 6.1 Test Scenarios and Results

Test Scenario	Result
Duplicate email registration rejected (HTTP 409)	PASS
Expired OTP rejected after 10 min (HTTP 400)	PASS

Booking on freelancer's unavailable date rejected (HTTP 409)	PASS
Invalid Razorpay signature rejected — no DB update (HTTP 400)	PASS
Non-admin accessing /api/admin/* blocked (HTTP 403)	PASS
Self-booking prevention (HTTP 400)	PASS
Geospatial query with (0,0) coordinates rejected (HTTP 400)	PASS
Field no-show auto-cancellation after 6 h (cron test)	PASS
Dispute SLA auto-resolution after 5 days (cron test)	PASS
Socket.IO message delivery after network reconnect	PASS
Capacitor StatusBar edge-to-edge fix on Android 15	PASS
Razorpay checkout in Capacitor WebView (Permissions-Policy fix)	PASS
Dark mode persists across sessions (localStorage)	PASS
FCM push delivered to background Android app	PASS
Portfolio deletion removes Cloudinary asset (destroy() called)	PASS
Duplicate review on same booking blocked (unique index, HTTP 409)	PASS

Table 6.1 Test Scenarios and Results

7 CONCLUSION & FUTURE WORK

7.1 CONCLUSION

PocketLancer successfully demonstrates the design and deployment of a full stack freelance and home services marketplace. The system integrates multiple functional modules across web and mobile platforms, supporting different user roles with appropriate interfaces and workflows. The platform is deployed live and includes both web access and a native Android application, showcasing a complete end-to-end solution.

A key contribution of the project is the dual-service-category architecture, which supports both digital freelancers through milestone-based escrow payments and field service workers through a trust-based Honor Score system. This unified approach addresses a gap in existing platforms. The system also incorporates efficient geospatial search, real-time communication, and secure transaction handling. Overall, the project demonstrates strong understanding of full-stack development, system design, and cloud deployment, exceeding typical academic project expectations.

7.2 FUTURE WORK

The system can be enhanced by integrating AI-based search and recommendation to provide more accurate and personalized results for users. Real-time location tracking for field services can improve transparency and user confidence during service delivery. Additionally, developing a fully native mobile application can enhance performance and enable advanced device-level features.

7.2.1 AI-Powered Semantic Search and Recommendation

The system can be further enhanced by integrating AI-based semantic search and recommendation systems. By using vector embeddings and similarity-based matching, the platform can provide more accurate freelancer recommendations beyond keyword search.

7.2.2 Live Freelancer Location Tracking

Real-time freelancer location tracking for field services can also be introduced to improve transparency and user confidence. This would allow clients to track service providers similar to ride-sharing applications.

7.2.3 React Native Rebuild

A future improvement includes rebuilding the mobile application using React Native to achieve deeper native integration, improved performance, and better compliance with mobile platform policies.

7.2.4 Subscription and Commission Model

Additionally, a subscription and commission-based monetization model can be implemented, offering different tiers for clients and freelancers with advanced features such as analytics, priority listing, and unlimited usage.

8 REFERENCES

- [1] J. J. Horton and L. B. Chilton, “The online laboratory: Conducting experiments in a real labor market,” *Proc. ACM EC*, pp. 71–80, 2010.
<https://doi.org/10.1145/1807342.1807369>
- [2] M. Kokkodis and P. G. Ipeirotis, “Reputation transferability in online labor markets,” *Management Science*, vol. 62, no. 6, pp. 1687–1706, 2016.
<https://doi.org/10.1287/mnsc.2015.2217>
- [3] P. Resnick et al., “Reputation systems,” *Communications of the ACM*, vol. 43, no. 12, pp. 45–48, 2000.
<https://doi.org/10.1145/355112.355122>
- [4] M. Babaioff et al., “Combinatorial agency,” *Journal of Economic Theory*, vol. 147, no. 3, pp. 999–1034, 2012.
- [5] NITI Aayog, “India’s Booming Gig and Platform Economy,” Government of India, 2022.
<https://www.niti.gov.in/>
- [6] I. Fette and A. Melnikov, “The WebSocket Protocol,” *IETF RFC 6455*, 2011.
<https://www.rfc-editor.org/rfc/rfc6455>
- [7] MongoDB, Inc., “Geospatial Queries,” *MongoDB Manual*, 2023.
<https://www.mongodb.com/docs/manual/geospatial-queries/>
- [8] Razorpay, “Razorpay Payment Gateway API Reference,” 2023.
<https://razorpay.com/docs/api/>
- [9] Ionic Framework, “Capacitor Documentation,” 2023.
<https://capacitorjs.com/>
- [10] Google Firebase, “Cloud Messaging Documentation,” 2023.
<https://firebase.google.com/docs/cloud-messaging>